SIEMENS

Functions for accessing Teamcenter Gateway 1

Functions for Connections 2

Functions on libraries 3

TIA Portal Openness: API for automation of engineering workflows

Manual

21.00.00.00

12/2025

NOTICE

This document was downloaded from the following site:

https://docs.industrial-operations-x.siemens.cloud

Siemens assumes no liability for the accuracy, completeness, or timeliness of the data provided.

Use is at your own risk.

Legal information

Warning notice system

This manual contains notices you have to observe in order to ensure your personal safety, as well as to prevent damage to property. The notices referring to your personal safety are highlighted in the manual by a safety alert symbol, notices referring only to property damage have no safety alert symbol. These notices shown below are graded according to the degree of danger.

DANGER

indicates that death or severe personal injury **will** result if proper precautions are not taken.

WARNING

indicates that death or severe personal injury **may** result if proper precautions are not taken.

CAUTION

indicates that minor personal injury can result if proper precautions are not taken.

NOTICE

indicates that property damage can result if proper precautions are not taken.

If more than one degree of danger is present, the warning notice representing the highest degree of danger will be used. A notice warning of injury to persons with a safety alert symbol may also include a warning relating to property damage.

Qualified Personnel

The product/system described in this documentation may be operated only by **personnel qualified** for the specific task in accordance with the relevant documentation, in particular its warning notices and safety instructions. Qualified personnel are those who, based on their training and experience, are capable of identifying risks and avoiding potential hazards when working with these products/systems.

Proper use of Siemens products

Note the following:

WARNING

Siemens products may only be used for the applications described in the catalog and in the relevant technical documentation. If products and components from other manufacturers are used, these must be recommended or approved by Siemens. Proper transport, storage, installation, assembly, commissioning, operation and maintenance are required to ensure that the products operate safely and without any problems. The permissible ambient conditions must be complied with. The information in the relevant documentation must be observed.

Trademarks

All names identified by ® are registered trademarks of Siemens Aktiengesellschaft. The remaining trademarks in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owner.

Disclaimer of Liability

We have reviewed the contents of this publication to ensure consistency with the hardware and software described. Since variance cannot be precluded entirely, we cannot guarantee full consistency. However, the information in this publication is reviewed regularly and any necessary corrections are included in subsequent editions.

Siemens Aktiengesellschaft
Digital Industries
Postfach 48 48
90026 NÜRNBERG
GERMANY

12/2025 Subject to change

Copyright © Siemens 2026.
All rights reserved

Table of contents

1 Functions for accessing Teamcenter Gateway 7
1.1 Connecting to the Teamcenter Gateway 7
1.2 Disconnecting to the Teamcenter Gateway 9
1.3 Connecting to the ConnectSSO 11
1.4 Saving Project/Global library in Teamcenter 13
1.5 Saving Project/Global library with proxy object 16
1.6 Saving Project/Global library with new item 19
1.7 Accessing custom attributes in Teamcenter 21
1.8 Saving Project to Teamcenter as new item 26
1.9 Saving project using save to item with proxy object 30
1.10 Saving Project as new item with proxy object 33
1.11 Saving project using save as new revision with proxy object 39
1.12 Saving Project to Teamcenter as new revision 41
1.13 Downloading Project/Global library from Teamcenter 44
1.14 Searching Project / Global library from Teamcenter 47
1.15 Checkout Project/Global library dataset in Teamcenter 50
1.16 Checkin Project/Global library dataset in Teamcenter 53
1.17 Cancel checkout dataset in Teamcenter 56

2 Functions for Connections 59
2.1 Configurable attributes of a port-to-port connection 59

3 Functions on libraries 64
3.1 Functions for objects and instances 64
3.2 Accessing global libraries 65
3.3 Accessing library base type 70
3.4 Accessing global library languages 71
3.5 Opening libraries 74
3.6 Comparing libraries 75
3.7 Enumerating open libraries 80
3.8 Saving and closing libraries 81
3.9 Archiving and retrieving a library 83
3.10 Creating global libraries 87
3.11 Accessing folders in a library 88
3.12 Accessing types 92
3.13 Accessing type versions 96
3.14 Accessing blocks located in library 104
3.15 Accessing instances 106
3.16 Accessing master copies 109
3.17 Create master copy from a project in library 112

3.18 Cleaning up library 114
3.19 Create an object from a master copy 118
3.20 Copying master copies 121
3.21 Creating new type from document 121
3.22 Updating type from document 129
3.23 Determining out-of-date type instances 136
3.24 Updating the project 140
3.25 Updating a library 142
3.26 Cleaning up library 144
3.27 Harmonize project from project library 145
3.28 Updating Structure during Library Update 149
3.29 Updating libraries type 160
3.30 Deleting library content 161
3.31 Setting default version of a type 164
3.32 Getting default version of a type 165
3.33 Improved types consistency state 166
3.34 Editing library type version 169
3.35 Discarding library type version 172
3.36 Managing alarm text list in PLC and Mastercopy library 174
3.37 Managing DoNotUse in Library 177
3.38 Exporting library type version as document 178
3.39 Retrieving supported export formats for library 182
3.40 Releasing library type version 183

Functions for accessing Teamcenter Gateway 1

1.1 Connecting to the Teamcenter Gateway

Requirement

- The TIA Portal application is connected to the TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Application

You can use the TIA Portal Openness to establish a connection with Teamcenter via TcGateway.

Parameter

Parameter name	Data type	Mandatory	Description
userName	System.String	Yes	Specifies the username in Teamcenter
userPassword	System.SecureString	Yes	Specifies the password of user in Teamcenter
serverURL	System.String	Yes	Specifies the fully qualified URL of Teamcenter server
teamcenterInstance	System.String	Yes	Specifies the instance of Teamcenter server
userRole	System.String	No	Specifies the user role

			defined in Teamcenter.
userGroup	System.String	No	Specifies the group to which the user belongs in Teamcenter.

Note

A Siemens.Engineering exception of type TcGatewayException is thrown, if any of the above mandatory parameter is not passed.

A successful connection returns a encrypted TcGatewayConnectionInfo object. In case the connection is not successful, an engineering exception of type TcGatewayException is thrown containing the reason for failure.

Data type	Description
TcGatewayConnectionInfo	The TcGatewayConnection Info object has to be mandatorily passed in all the TcGateway openness APIs. It is used in other TcGateway Openness APIs to validate the source of the call and to validate the active Teamcenter session. The TcGatewayConnectionInfo provides the following connection details: • SessionToken (active connection session token) • Group (Teamcenter user group) • Role (Teamcenter user role)

Program code

```
Copies the following program code to clipboard.
private void ConnectToTeamcenter(SecureString password)
{
    // Refer the Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    // Retrieves the Teamcenter Gateway connection provider
    // from the TiaPortal instance using the GetService
method.
```

```
    TeamcenterConnectionProvider tcGatewayProvider =
tiaPortal.GetService<TeamcenterConnectionProvider>();
    // Replace with actual username
    string userName = "yourUserName";
    // Replace with actual role
    string userRole = "yourRole";
    // Replace with actual group
    string userGroup = "yourGroup";
    // Replace with actual Teamcenter server URL
    string serverURL = "yourServerURL";
    // Replace with actual instance
    string teamcenterInstance = "yourInstance";
    //Establish a connection to the Teamcenter using provided
credentials
    TcGatewayConnectionInfo connectionInfo =
tcGatewayProvider.Connect(userName, password, userGroup,
    userRole, serverURL, teamcenterInstance);
    // Represents the group information retrieved from the
Teamcenter connection
    string group = connectionInfo.Group;
    // Represents the role information retrieved from the
Teamcenter connection
    string role = connectionInfo.Role;
}
```

1.2 Disconnecting to the Teamcenter Gateway

Requirement

- The TIA Portal application is connected to the TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Application

You can use the TIA Portal Openness to disconnect the connection between the Teamcenter server and TIA Portal using TcGateway.

Parameter

Parameter	Data type	Mandatory	Description
tcGatewayConnectionInfo	TcGatewayConnectionInfo	Yes	Returns the TcGatewayConnectionInfo object during Connect / ConnectSSO API call.

Program code



```
Copies the following program code to clipboard.
private void DisconnectToTeamCenter(SecureString password)
{
    // Refer the Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    // Retrieves the Teamcenter Gateway connection provider
    // from the TiaPortal instance using the GetService
method.
    TeamcenterConnectionProvider tcGatewayProvider =
tiaPortal.GetService<TeamcenterConnectionProvider>();
    //Replace with username
    string userName = "yourUserName";
    // Replace with user role
    string userRole = "yourUserRole";
    //Replace with user group
    string userGroup = "yourUserGroup";
    //Teamcenter server URL
    string serverURL = "yourServerURL";
    //Replace with Teamcenter Instance
    string teamcenterInstance = "yourInstance";
```

```
    //Establish a connection to Teamcenter using provided
credentials
    TcGatewayConnectionInfo tcgConnectionInfo =
    tcGatewayProvider.Connect(userName,
password,serverURL,teamcenterInstance, userRole, userGroup);
    // Disconnect to Teamcenter
    tcGatewayProvider.Disconnect(tcgConnectionInfo);
}
```

See also

Connecting to the Teamcenter Gateway

1.3 Connecting to the ConnectSSO

Requirement

- The TIA Portal application is connected to the TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Application

You can use the TIA Portal Openness to call the ConnectSSO() if an active SSO session is available with below required parameters.

Parameter	Data type	Mandatory	Description
hostURL	System.String	Yes	Specifies the fully qualified Teamcenter server host with port number.
instance	System.String	Yes	Specifies the Teamcenter server instance.

loginURL	System.String	Yes	Specifies the URL for the Login Service configured as part of Teamcenter Security Services.
applicationID	System.String	Yes	Specifies the Application ID configured in Teamcenter Application registry via Identity service

A successful connection returns an encrypted TcGatewayConnectionInfo object. In case the connection is not successful, an appropriate exception is thrown.

Note

If no active SSO session is available, you will be prompted to enter Teamcenter credentials in browser to create SSO session. The API supports secure card based login via PKI.

Program code



```
Copies the following program code to clipboard.
//Program code: ConnectSSO using TcGatewayConnectionInfo
private void ConnectingToTheConnectSSO()
{
    // Refer the Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    // Retrieves the Teamcenter Gateway connection provider
    // from the TiaPortal instance using the GetService method.
    TeamcenterConnectionProvider tcGatewayProvider =
tiaPortal.GetService<TeamcenterConnectionProvider>();
    // Replace with actual values
    string hostUrl = "host_url";
    string teamcenterInstance = "teamcenter_instance";
    string loginURL = "teamcenter SSO_login_service_url";
    String applicationId = "teamcenter_SSO_application";
    // Connect to Teamcenter SSO
    TcGatewayConnectionInfo connectionInfo =
tcGatewayProvider.ConnectSSO(hostUrl,teamcenterInstance,loginURL,applica
```

```
    // Retrieve connection details
    string group = connectionInfo.Group;
    string role = connectionInfo.Role;
}
```

1.4 Saving Project/Global library in Teamcenter

Requirement

- The TIA Portal application is connected to the TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to save a project/global library via the TcGatewayWorkflowProvider service. This service will be available at Project / Global Library composition.

Parameter

Parameter name	Data type	Is Mandatory	Description
tcGatewayConnectionInfo	TcGatewayConnectionInfo	Yes	Returns the tcGatewayConnectionInfo during connection to Teamcenter. The tcGatewayConnectionInfo is passed to match with the active connection's tcGatewayConnectionInfo .
localCacheOption	LocalCacheOption(Enum)	Yes	TcGateway LocalCacheOption to overwrite the TIA project/ Global library in Teamcenter cache.

.

Datatype	Description
Siemens.Engineering.TeamcenterGateway.ItemInfo	The API returns the following ItemInfo object after th or global library is successfully saved to the Teamcer • ItemId [System.String] • RevisionId [System.String] • ItemName [System.String] • ItemType [Siemens.Engineering.TeamcenterGateway.ItemTy

Program code

 Copies the following program code to clipboard.

```
//Program code: Saving Project/gobal library in Teamcenter using Save()
private void SaveProjectInTeamCenter(SecureString password)
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    TeamcenterConnectionProvider tcGatewayConnectionProvider = tiaPortal.GetService<TeamcenterConnectionProvider>();
    TcGatewayLockProvider tcGatewayLockProvider = tiaPortal.GetService<TcGatewayLockProvider>();
    try
    {
    // Define user-specific credentials and connection details.
    string userName = "name";
    string userRole = "role";
    string userGroup = "group";
    string serverURL = "teamcenter_server_url";
    string teamcenterInstance = "instance";
    // Connect to Teamcenter using the provided credentials
```

```
    TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.Connect(userName, password,
    userGroup, userRole, serverURL, teamcenterInstance);
    // Checkout a specific dataset from Teamcenter for
editing.
    tcGatewayLockProvider.CheckoutDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");
    // Access the project composition in the TiaPortal
instance
    ProjectComposition projectComposition =
tiaPortal.Projects;
    // Define the project path for the dataset checked out
earlier
    // or download it using the TcGateway API.
    string projectPath = "project_path";
    // Open the project using the specified project path.
    Project tcgProject = projectComposition.Open(new
FileInfo(projectPath));
    // Create a device group within the project.
    tcgProject.DeviceGroups.Create("Group_1");
    // Save the project locally in the TC cache (optional
step).
    tcgProject.Save();
    // Access the TcGatewayWorkflowProvider from the project
    // instance using GetService()
    TcGatewayWorkflowProvider tcgWorkflowProvider =
tcgProject.GetService<TcGatewayWorkflowProvider>();
    // Choose an appropriate local cache option
    // and save the changes made to the project.
    LocalCacheOption cacheOption =
LocalCacheOption.Overwrite;
    // Choose the appropriate option
    ItemInfo response =
tcgWorkflowProvider.Save(connectionInfo, cacheOption);
```

```
    // Check in the dataset back to Teamcenter after making
changes.
    tcGatewayLockProvider.CheckinDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");
    }
    catch (TcGatewayException ex)
    {
     // Handle exception or log error
    }
}
```

1.5 Saving Project/Global library with proxy object

Requirement

- The TIA Portal application is connected to the TIA Portal Openness
  See Connection to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to archive the Project / Global library and then save the archive object to the Teamcenter dataset along with proxy objects (e.g. Plc, Program blocks, UDTs, other supported types).

Once the project / Global Library is successfully saved to Teamcenter, the API return ItemInfo object containing ItemId, RevisionId, ItemName and ItemType details.

Parameter

Parameter name	Data type	Is Mandatory	Description
tcGatewayConnectionInfo	TcGatewayConnectionInfo	Yes	Returns the tcGatewayConnectionInfo during connection to Teamcenter. The tcGatewayConnectionInfo has passed has to match with the active

			connection's tcGatewayConnectionInfo
localCacheOption	LocalCacheOption(Enum)	Yes	TcGateway LocalCacheOption to overwrite TIA project/ Global library in TC cache.

Program code:



```
Copies the following program code to clipboard.
//Program code: Saving project/ global library in Teamcenter using SaveWithProxyObject()
private void SaveProjectWithProxyObject(SecureString password)
{
    // Refer the Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    TeamcenterConnectionProvider tcGatewayConnectionProvider = tiaPortal.GetService<TeamcenterConnectionProvider>();
    TcGatewayLockProvider tcGatewayLockProvider = tiaPortal.GetService<TcGatewayLockProvider>();
    try
    {
   // Define user-specific credentials and connection details.
    string userName = "your_username";
    string userRole = "your_userrole";
    string userGroup = "your_usergroup";
    string serverURL = "serverURL";
    string teamcenterInstance = "teamcenterInstance";
    // Connect to Teamcenter using the provided credentials
    TcGatewayConnectionInfo connectionInfo = tcGatewayConnectionProvider.Connect
    (userName, password, userGroup, userRole, serverURL, teamcenterInstance);
```

```
    // Checkout a specific dataset from Teamcenter for
editing.
    tcGatewayLockProvider.CheckoutDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");
    // Access the project composition in the TiaPortal
instance
    ProjectComposition projectComposition =
tiaPortal.Projects;
    // Define theProject path for the checkout dataset
    // in above step or download project using TcGateway Load
API
    string projectPath = "projectPath";
    // Open the project using the specified project path.
    Project tcgProject = projectComposition.Open(new
FileInfo(projectPath));
    // Create a device group within the project.
    TcGatewayWorkflowProvider tcgWorkflowProvider =
tcgProject.GetService<TcGatewayWorkflowProvider>();
    // Save the project with proxy object, choose appropriate
LocalCacheOption
    ItemInfo responseWithOverwrite =
tcgWorkflowProvider.SaveWithProxyObject(connectionInfo,
LocalCacheOption.Overwrite);
    //OR
    ItemInfo responseWithoutOverwrite =
tcgWorkflowProvider.SaveWithProxyObject(connectionInfo,
LocalCacheOption.DoNotOverwrite);
    // Check in the dataset after operations
    tcGatewayLockProvider.CheckinDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");
    }
    catch (TcGatewayException ex)
    {
    // Handle TcGatewayException or perform other operations
```

```
        }
}
```

See also

Connecting to the TIA Portal

1.6 Saving Project/Global library with new item

Requirement

- Connecting to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Parameter

Parameter name	Data type	Is Manc
tcGatewayConnectionInfo	Siemens.Engineering.TeamcenterGateway.TcGatewayConnectionInfo	Yes
itemId	System.String	Yes
revisionId	System.String	Yes
localCacheOption	Siemens.Engineering.TeamcenterGateway.LocalCacheOption(Enum)	Yes

			dataset version of project mismatches with server project version

Program code



```
Copies the following program code to clipboard.

//Program code: Saving project/global library in Teamcenter
using SaveToItem( )

private void SavingProjectWithSaveToItem(SecureString
password)

{

    // Refer Connecting to the TIA Portal section

    TiaPortal tiaPortal = new TiaPortal();

    // Retrieve TcGatewayConnectionProvider from
GetService()using tiaPortal instance

    TeamcenterConnectionProvider tcGatewayConnectionProvider
= tiaPortal.GetService<TeamcenterConnectionProvider>();

    // Retrieve TcGatewayLockProvider from GetService() using
tiaPortal instance

    TcGatewayLockProvider tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();

    try

    {

    // Define user-specific credentials and connection
details

    string userName = "your_userName";

    string userRole = "your_role";

    string userGroup = "your_group";

    String serverURL = "serverURL";

    String teamcenterInstance = "teamcenterInstance";

    // Connect to Teamcenter using the provided credentials

    TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.Connect(userName, password,
userGroup,

    userRole, serverURL, teamcenterInstance);
```

```
    tcGatewayLockProvider.CheckoutDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");
    ProjectComposition projectComposition =
tiaPortal.Projects;
    // Define Project path for the checkout dataset in above
step
    // or download project using TcGateway Download API
    String projectPath = "projectPath";
    // Open the project using the specified project path
    Project tcgProject = projectComposition.Open(new
FileInfo(projectPath));
    // Access the TcGatewayWorkflowProvider from the project
    // instance using GetService()
    TcGatewayWorkflowProvider tcgWorkflowProvider =
tcgProject.GetService<TcGatewayWorkflowProvider>();
    // Choose an appropriate local cache option
    // and save the changes made to the project.
    ItemInfo response =
tcgWorkflowProvider.SaveToItem(connectionInfo, "000495", "A",
LocalCacheOption.Overwrite);
    // Check in the dataset back to Teamcenter after making
changes.
    tcGatewayLockProvider.CheckinDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");
    }
    catch (TcGatewayException ex)
    {
     // Handle exception or log error or other operations
    }
}
```

1.7 Accessing custom attributes in Teamcenter

Requirement

- The TIA Portal Openness is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Intriduction

You can use the TIA Portal Openness to get and set the custom attributes value in the Teamcenter Gateway.

Parameter

The Teamcenter itemtype refers to T4TiaLibrary,T4TiaProject and subtypes which are present in Teamcenter Datamodel for which custom attributes are mapped in Teamcenter.

Parameter name	Data type	Is Manc
tcGatewayConnectionInfo	Siemens.Engineering.TeamcenterGateway.TcGatewayConnectionInfo	Yes
itemType	System.String	Yes

After successful execution, the API returns collection of custom attributes if available in the Teamcenter for given itemtype.

Data type	Description
IList<Siemens.Engineering.TeamcenterGateway.TeamcenterProperty>	Specifies the list of Teamcenter cu • Name [System.String] • MappingLevel [System.int] • ListOfValueInfo [Siemens.Engineering.Teamcer - LowerLimit [System.String]

	- UpperLimit [System.String] - Values [System.Collections.Generic.IEnumerable<System.String>] • DataType [Siemens.Engineering.TeamcenterGateway.MappedCustomAttributeType(Enum)]

The Call SetValue API to set value on custom attributes fetched using GetTeamcenterCustomAttributes API along with pass error callback delegate.

Parameter name	Data type	Is Mandatory	Description
value	System.String	Yes	Specifies the valu for custom attribute
errorCallback	Siemens.Engineering.TeamcenterGateway.ErrorCallback	Yes	Delegate to handle error which are raised during API invocation/actior

Program code



```
Copies the following program code to clipboard.

private void AccessCustomattributes(SecureString password)
{
    // Refer the Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    var tcGatewayConnectionProvider =
tiaPortal.GetService<TeamcenterConnectionProvider>();
    var tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();
    try
    {
    // Define user-specific credentials information
    string userName = string.Empty;
    string userRole = "your_role";
```

```
    string userGroup = "your_group";
    string serverURL = "serverURL";
    string teamcenterInstance = "teamcenterInstance";
    // Delegate for item details
    ItemDetailsDelegate itemDetailsDelegate = ItemDetailsForCreation;
    void ItemDetailsForCreation(ItemDetails itemDetails)
    {
    itemDetails.ItemId = "...";
    itemDetails.ItemName = "Project30";
    itemDetails.RevisionId = "...";
    itemDetails.TeamcenterItemType = "T4TiaProject";
    itemDetails.Comment = "Project Comments.";
    }
    // Connect to Teamcenter using provided credentials
    TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.Connect(userName, password, userGroup,
    userRole, serverURL, teamcenterInstance);
    // Open project or global library in TIA Portal
    var project = tiaPortal.Projects.Open(new
FileInfo(@"~\UserData\Documents\Automation\Project30\Project30.ap18"));
    var tcGatewayWorkflowProviderForProject =
project.GetService<TcGatewayWorkflowProvider>();
    //OR
    var library = tiaPortal.GlobalLibraries.Open(new
FileInfo(@"~\Documents\Automation\Library7\Library7.al18"),
OpenMode.ReadWrite);
    // Get the workflow provider for the project or library
    var tcGatewayWorkflowProviderForLibrary =
library.GetService<TcGatewayWorkflowProvider>();
    // Get the custom attributes from Teamcenter Project
    var attributes = tcGatewayWorkflowProviderForProject.
    GetTeamcenterCustomAttributes(connectionInfo,"T4TiaProject");
    // Error callback for property value setting
```

```
    ErrorCallback errorCallback =
PropertyValueSettingErrorCallback;
    void PropertyValueSettingErrorCallback(string
errorMessage)
    {
    Console.WriteLine(errorMessage);
    }
    // Loop through attributes and update values
    foreach (var arr in attributes)
    {
        // Display values from ListOfValueInfo if available
        if (arr.ListOfValueInfo != null)
        {
            foreach (var l in arr.ListOfValueInfo.Values)
            {
            Console.WriteLine(l);
            }
        }
        switch (arr.DataType)
        {
        case MappedCustomAttributeType.String:
        arr.SetValue("23232323232", errorCallback);
        break;
        case MappedCustomAttributeType.Char:
        arr.SetValue("d", errorCallback);
        break;
        case MappedCustomAttributeType.Integer:
        arr.SetValue("232", errorCallback);
        break;
        case MappedCustomAttributeType.Float:
        arr.SetValue(23.4f.ToString(), errorCallback);
        break;
```

```
        }
      }
     }
     catch (TcGatewayException ex)
     {
      // Handle exception or log error or other operations or
other operations
     }
}
```

1.8 Saving Project to Teamcenter as new item

Requirement

- The TIA Portal Openness is connected to the TIA Portal
  See Connecting to the TIA Portal
- A Project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to save a TIA Portal project in Teamcenter as a new item. The API returns itemInfo object containing the ItemId, RevisionId, ItemType, ItemName after successful creation of item/revision.

Parameter

Parameter name	Data type	Is Mandatory	Description

tcGatewayConnectionInfo	Siemens.Engineering.TeamcenterGateway.TcGatewayConnectionInfo
itemDetailsDelegate	delegate
customAttributes	IEnumerable<Siemens.Engineering.TeamcenterGateway.TeamcenterProperty>

Program code

```
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    TeamcenterConnectionProvider tcGatewayConnectionProvider =
tiaPortal.GetService<TeamcenterConnectionProvider>();
    TcGatewayLockProvider tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();
    try
    {
        // Define user specific credentials and connection details
        String userName = string.Empty;
        String userRole = "your_role";
        String userGroup = "your_userGroup";
        String serverURL = "serverURL";
        String teamcenterInstance = "teamcenterInstance";
        // Connect to Teamcenter using the provided credentials
        TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.Connect(userName, password,
        userGroup, userRole, serverURL, teamcenterInstance);
        var project = tiaPortal.Projects.Open(new
        FileInfo(@"D:\\UserData\\Documents\\Automation\\Project30\\Proje
        var library = tiaPortal.GlobalLibraries.Open(new
FileInfo(@"~\Documents\Automation\Library7\Library7.al18"),
        OpenMode.ReadWrite);
        var tcGatewayWorkflowProvider = project.GetService<TcGatewayWorl
();
        var tcGatewayWorkflowProviderForLibrary =
library.GetService<TcGatewayWorkflowProvider>();
        // Define item details
        ItemDetailsDelegate itemDetailsDelegate = (itemDetails) =>
        {
            itemDetails.ItemId = "...";
```

```
            itemDetails.ItemName = "Proj_Object";
itemDetails.RevisionId = "...";
            itemDetails.TeamcenterItemType = "T4TiaProject";
            itemDetails.Comment = "Prooject Comments.";
            itemDetails.TeamcenterFolder =
"Home\\TIA_Project\\ConveyourBelt";
            itemDetails.TeamcenterProject = new string[]
{"Project_TIA_1", "Project_TIA_2"};
         };
        // Define error callback
        ErrorCallback errorCallback = (errorMessage) =>
        {
            Console.WriteLine(errorMessage);
        };
        //Save item in Teamcenter with custom attributes.
        var attributes =
tcGatewayWorkflowProvider.GetTeamcenterCustomAttributes(connectionInfo,
"T4TiaProject");
        foreach (var arr in attributes)
        {
            if (arr.ListOfValueInfo != null)
            {
                foreach (var l in arr.ListOfValueInfo?.Values)
                {
                  Console.WriteLine(l);
                }
            }
            switch (arr.DataType)
            {
                case MappedCustomAttributeType.String:
                arr.SetValue("23232323232", errorCallback);
                break;
```

```
                case MappedCustomAttributeType.Char:
                arr.SetValue("d", errorCallback);
                break;
                case MappedCustomAttributeType.Integer:
                arr.SetValue("232", errorCallback);
                break;
                case MappedCustomAttributeType.Float:
                arr.SetValue(23.4f.ToString(), errorCallback);
                break;
            }
            var createdItem =
tcGatewayWorkflowProvider.SaveAsNewItem(connectionInfo,
itemDetailsDelegate, attributes);
            var itemId = createdItem.ItemId;
            var revisionId = createdItem.RevisionId;
            var itemName = createdItem.ItemName;
            var type = createdItem.ItemType;
            //Checkout created dataset to edit in TIA portal
            tcGatewayLockProvider.CheckoutDataset(connectionInfo,
itemId, revisionId, DatasetType.T4TiaProjectDataset, itemName);
            //Check in dataset after changes to project are
completed
            tcGatewayLockProvider.CheckinDataset(connectionInfo,
itemId, revisionId, DatasetType.T4TiaProjectDataset, itemName);
        }
    }
    catch (TcGatewayException ex)
    {
     // Handle exception
    }
}
```

1.9 Saving project using save to item with proxy object

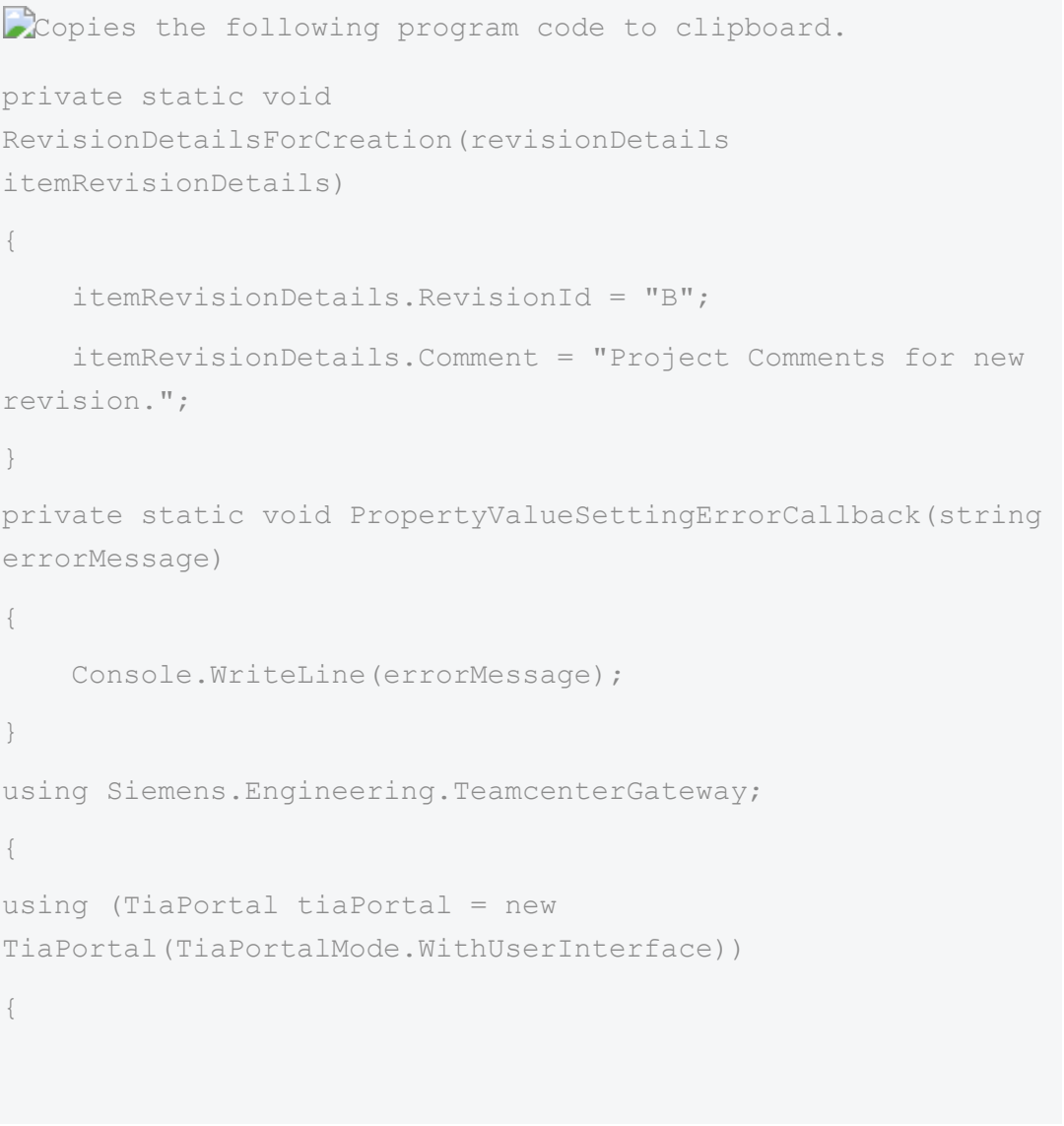
Requirement

- The TIA Portal Openness is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness application to to save a TIA Portal Project with proxy objects into Teamcenter as a new Item revision. The API returns Siemens.Engineering.TeamcenterGateway.ItemInfo.

Program code



```
    String userName = ...//User name
    SecureString password = ...;
    //Secure string.
    // Client takes care of encrypting password and openness framework
    // decrypts and passes the decrypted value to the action;
    String userRole = null;//Role
    String userGroup = null//Group
    String serverURL = ...//Teamcenter server URL
    String teamcenterInstance=....//Instance
    RevisionDetailsDelegate revisionDetailsDelegate = RevisionDetailsFo
    TcGatewayConnectionInfo connectionInfo = tcGatewayConnectionProvide
    (userName, password, userGroup, userRole, serverURL,teamcenterInsta
     var project = tiaPortal.Projects.Open
    (new
FileInfo("D:\\UserData\\Documents\\Automation\\tc\Project30\\Project30.a
    //OR
    //var library = tiaPortal.GlobalLibraries.Open
    (new FileInfo(@"~\Documents\Automation\Library7\Library7.al18"),
OpenMode.ReadWrite);
    //var tcGatewayWorkflowProvider = library.GetService<TcGatewayWorkfl
    var tcGatewayWorkflowProvider = project.GetService<TcGatewayWorkflo
    //Save item in Teamcenter with custom attributes.
    var attributes =
tcGatewayWorkflowProvider.GetTeamcenterCustomAttributes(connectionInfo,"
    ErrorCallback errorCallback = PropertyValueSettingErrorCallback;
    foreach (var arr in attributes)
    {
    if (arr.ListOfValueInfo != null){
    foreach (var l in arr.ListOfValueInfo?.LovValues)
    {
    Console.WriteLine(l);
}
```

```
    }
    switch (arr.DataType)
    {
    case
MappedCustomAttributeType.String:arr.SetValue("23232323232",
errorCallback);
    break;
    case MappedCustomAttributeType.Char:
    arr.SetValue("d", errorCallback);
    break;
    case MappedCustomAttributeType.Integer:
    arr.SetValue("232", errorCallback);
    break;
    case MappedCustomAttributeType.Float:
    arr.SetValue(23.4f.ToString(), errorCallback);
    break;
    }
    var createdItemRevision =
wf.SaveAsNewRevisionWithProxyObject
    (connectionInfo, RevisionDetailsDelegate, attributes);
    //OR
    //var createdItemRevision =
wf.SaveAsNewRevisionWithProxyObject(connectionInfo,
RevisionDetailsDelegate, null);<-- optional
    }
    catch(TcGatewayException)
    {
    // other operations
    }
    }
}
```

1.10 Saving Project as new item with proxy object

Requirement

- The TIA Portal Openness is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to save the TIA Portal project and Global library to the Team Center along with proxy object. The API returns itemInfo object details such as ItemId, RevisionId, ItemType, ItemName after successful creation of new item.

Parameter

Parameter name	Data type	Is Manc
tcGatewayConnectionInfo	Siemens.Engineering.TeamcenterGateway.TcGatewayConnectionInfo	Yes
itemDetailsDelegate	delegate	Yes

customAttributes	IEnumerable<Siemens.Engineering.TeamcenterGateway.TeamcenterProperty>	No

Program code



```
Copies the following program code to clipboard.
private void
SavingProjectAsNewItemWithProxyObject(SecureString password)
{
    // Refer the Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    // Retrieve the TcGatewayConnectionProvider and
TcGatewayLockProvider
    // from GetService() using tiaPortal instance
    TeamcenterConnectionProvider tcGatewayConnectionProvider
= tiaPortal.GetService<TeamcenterConnectionProvider>();
    TcGatewayLockProvider tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();
    try
    {
    // Define user-specific credentials and connection
        String userName = string.Empty;
        String userRole = "your_role";
```

```
        String teamcenterInstance = string.Empty;
   // Connection to the Teamcenter using user's credential
    TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.Connect(userName, password, userGroup,
    userRole, serverURL, teamcenterInstance);
    // Open the TIA Portal project file located at the specified path
    var project = tiaPortal.Projects.Open(new
    FileInfo(@"D:\UserData\Documents\Automation\Project30\Project30.ap18
    // Retrieve the workflow provider for the opened project to perform
    // Teamcenter-related operations
    var tcGatewayWorkflowProvider =
project.GetService<TcGatewayWorkflowProvider>();
    //OR
    var library = tiaPortal.GlobalLibraries.Open(new
FileInfo(@"~\Documents\Automation\Library7\Library7.al18"),
OpenMode.ReadWrite);
    var tcGatewayWorkflowProviderForLibrary =
library.GetService<TcGatewayWorkflowProvider>();
    ItemDetailsDelegate itemDetailsDelegate = (ItemDetails itemDetails)
    {
        itemDetails.ItemId = "...";
        itemDetails.ItemName = "Proj_Object";
        itemDetails.RevisionId = "...";
        itemDetails.TeamcenterItemType = "T4TiaProject";
        itemDetails.Comment = "Project Comments.";
        itemDetails.TeamcenterFolder = "Home\\TIA_Project\\ConveyourBelt
        itemDetails.TeamcenterProject = new string[] { "Project_TIA_1",
"Project_TIA_2" };
    };
    // Save item in Teamcenter with custom attributes.
    // Retrieve custom attributes for a Teamcenter item of type
"T4TiaProject"
    // using the workflow provider
```

```
    var attributes =
tcGatewayWorkflowProvider.GetTeamcenterCustomAttributes(connectionInfo,
"T4TiaProject");
    // Set up an error callback delegate for handling property value
setting errors
    ErrorCallback errorCallback = (string errorMessage) =>
Console.WriteLine(errorMessage);
    // Iterate through each attribute in the retrieved custom
attributes
    foreach (var arr in attributes)
    {
        // Display values from ListOfValueInfo if available
        if (arr.ListOfValueInfo != null)
        {
            foreach (var l in arr.ListOfValueInfo?.Values)
            {
            Console.WriteLine(l);
            }
         }
         switch (arr.DataType)
         {
          case MappedCustomAttributeType.String:
          arr.SetValue("23232323232", errorCallback);
          break;
          case MappedCustomAttributeType.Char:
          arr.SetValue("d", errorCallback);
          break;
          case MappedCustomAttributeType.Integer:
          arr.SetValue("232", errorCallback);
          break;
          case MappedCustomAttributeType.Float:
          arr.SetValue(23.4f.ToString(), errorCallback);
```

```
          break;
          }
         //Save the Teamcenter item as a new item with the specified
attributes
         // and proxy object
         var createdItem =
tcGatewayWorkflowProvider.SaveAsNewItemWithProxyObject(connectionInfo,
itemDetailsDelegate, attributes);
         //OR
         //< --optional
         /*var createdItem =
tcGatewayWorkflowProvider.SaveAsNewItem(connectionInfo,
itemDetailsDelegate, null);*/
         var itemId = createdItem.ItemId;
         var revisionId = createdItem.RevisionId;
         var itemName = createdItem.ItemName;
         var type = createdItem.ItemType;
        //Checkout created dataset to edit in TIA portal
        tcGatewayLockProvider.CheckoutDataset(connectionInfo, itemId,
revisionId, DatasetType.T4TiaProjectDataset,
        itemName);
        //Check in dataset after changes to project are completed
        tcGatewayLockProvider.CheckinDataset(connectionInfo, itemId,
revisionId, DatasetType.T4TiaProjectDataset,
        itemName);
      }
     }
     catch (TcGatewayException ex)
     {
     // other operations
     }
}
```

1.11 Saving project using save as new revision with proxy object

Requirement

- The TIA Portal Openness is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use TIA Portal Openness to save the modified TIA project and its internal data as proxy objects into a specific item/revision and dataset within a Teamcenter project, including the proxy objects. This operation returns ItemInfo.

Program code



```
Copies the following program code to clipboard.
using Siemens.Engineering.TeamcenterGateway;
private static void SaveProjectAsNewRevision()
{
using (TiaPortal tiaPortal = new
TiaPortal(TiaPortalMode.WithUserInterface))
{
    var tcGatewayConnectionProvider=
tiaPortal.GetService<TeamcenterConnectionProvider>();
    var tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();
    try
    {
    String userName = ...;//User name
    SecureString password = ...;
    //Secure string. Client takes care of encrypting password
    // and openness framework decrypts and passes the
decrypted value to the action;
    String userRole = null;//Role
    String userGroup = null;//Group
```

```
    String serverURL = ...;//Teamcenter server URL
    String teamcenterInstance=....;//Instance
    TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.
    Connect(userName, password, userGroup, userRole,
serverURL,teamcenterInstance);
    tcGatewayLockProvider.CheckoutDataset(connectionInfo ,
    "000495","B",DatasetType.T4TiaProjectDataset,"Project30");
    ProjectComposition projectComposition =
tiaPortal.Projects;
    String projectPath = ...;
    // Project path for the checkout dataset in above step
    // or download project using TcGateway Load API
    Project tcgProject = projectComposition.Open(projectPath);
    TcGatewayWorkflowProvider tcgWorkflowProvider =
tcgProject.GetService<TcGatewayWorkflowProvider>();
    ItemInfo response =
tcgWorkflowProvider.SaveToItemWithProxyObject(connectionInfo ,
"000495","B",
    LocalCacheOption.OverwriteLocalCache);
    //OR if client doesn't want to overwrite the local cache,
    // use DoNotOverwriteLocalCache of LocalCacheOption enum
    //ItemInfo response =
tcgWorkflowProvider.SaveToItemWithProxyObject(connectionInfo ,
"000495","B",
    // LocalCacheOption.DoNotOverwriteLocalCache);
    tcGatewayLockProvider.CheckinDataset(connectionInfo
,"000495","B",DatasetType.T4TiaProjectDataset,"Project30");
    }
    catch(TcGatewayException)
    {
    // other operations
    }
```

```
}
}
```

1.12 Saving Project to Teamcenter as new revision

Requirement

- The TIA Portal Openness is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Application

You can use the TIA Portal Openness to save the TIA Portal project and Global library in Teamcenter as new revision. The API returns the ItemInfo object such as ItemId, RevisionId, ItemName and ItemType details after successful creation of new revision.

Parameter

Parameter name	Data type
tcGatewayConnectionInfo	Siemens.Engineering.TeamcenterGateway.TcGatewayConnectionInfo
revisionDetailsDelegate	RevisionDetailsDelegate
customAttributes	IEnumerable<Siemens.Engineering.TeamcenterGateway.TeamcenterProperty

Program code



```
Copies the following program code to clipboard.
private void SavingProjectToTeamcenterAsNewRevision(SecureString
password)
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    var tcGatewayConnectionProvider =
tiaPortal.GetService<TeamcenterConnectionProvider>();
    var tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();
    try
    {
    String userName = "your_userName";//User name
    String userRole = "yourUserRole";//Role
    String userGroup = "yourUserGroup";//Group
    String serverURL = "serverURL";//Teamcenter server URL
    String teamcenterInstance =
"teamcenterInstance";//InstanceRevisionDetailsDelegate
    TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.Connect(userName, password,
    userGroup, userRole, serverURL, teamcenterInstance);
    var project = tiaPortal.Projects.Open(new
FileInfo(@"D:\UserData\Documents\Automation\Project30\Project30.ap18"));
    var tcGatewayWorkflowProvider =
project.GetService<TcGatewayWorkflowProvider>();
    ItemDetailsDelegate itemDetailsDelegate = ItemDetailsForCreation;
    void ItemDetailsForCreation(ItemDetails itemDetails)
    {
    itemDetails.ItemId = "...";
    itemDetails.ItemName = "Project30";
    itemDetails.RevisionId = "...";
```

```
    itemDetails.TeamcenterItemType = "T4TiaProject";
    itemDetails.Comment = "Project Comments.";
    }
    RevisionDetailsDelegate revisionDetailsDelegate =
RevisionDetailsForCreation;
    void RevisionDetailsForCreation(RevisionDetails
revisionDetails)
    {
    revisionDetails.RevisionId = "B";
    revisionDetails.Comment = "Revised project";
    }
    var saveAsNewItemResult =
tcGatewayWorkflowProvider.SaveAsNewItem(connectionInfo,
itemDetailsDelegate, null);
   //Checkout created dataset to edit in TIA portal
    tcGatewayLockProvider.CheckoutDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");
    var saveAsNewRevisionResult =
tcGatewayWorkflowProvider.SaveAsNewRevision(connectionInfo,
revisionDetailsDelegate, null);
    //Check in dataset after changes to project are complete
    tcGatewayLockProvider.CheckinDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");
   //Checkout revised project/library
    tcGatewayLockProvider.CheckoutDataset(connectionInfo,
"000495", "B", DatasetType.T4TiaProjectDataset, "Project30");
   //Check in revised dataset after changes to project are
complete
   tcGatewayLockProvider.CheckinDataset(connectionInfo,
"000495", "B", DatasetType.T4TiaProjectDataset, "Project30");
   }
   catch (TcGatewayException)
   {
   // other operations
```

```
    }
}
```

1.13 Downloading Project/Global library from Teamcenter

Requirement

- The TIA Portal Openness is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to download and open the project and library from Teamcenter. The API returns starter file path to open the project or library in the TIA Portal.

Parameter

Parameter name	Data type	Is Manc
tcGatewayConnectionInfo	Siemens.Engineering.TeamcenterGateway.TcGatewayConnectionInfo	Yes
itemID	System.String	Yes
revisionID	System.String	Yes

itemType	Siemens.Engineering.TeamcenterGateway.ItemType(Enum)	Yes	Item typ Project/L in Teamcer
localCacheOption	Siemens.Engineering.TeamcenterGateway.LocalCacheOption(Enum	Yes	Overwri contents TcCache incase a version Project / Global L being downloa is alread available True me overwrit content. Incase o False a exceptio thrown i there is version mismato betweer TcCache Teamcer or if the contents been modifie

Note

If any of the mandatory parameter is not passed, a Siemens.Engineering exception of type TcGatewayException is thrown.

Program code



```
private void DownloadProjectLibrary(SecureString password)
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    // Retrieve TcGatewayConnectionProvider and
TcGatewayLockProvider from GetService()
    // using tiaPortal instance
    TeamcenterConnectionProvider tcGatewayConnectionProvider=
tiaPortal.GetService<TeamcenterConnectionProvider>();
    TcGatewayLockProvider tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();
    // Define user-specific credentials and connection
    string userName = "userName";
    string userRole = "userRole";
    string userGroup = "userGroup";
    string serverURL = "TeamcenterserverURL";
    string teamcenterInstance = "teamcenterInstance";
    try
    {
    // Connection to the Teamcenter using user's credential
    TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.Connect(userName, password,
userGroup, userRole, serverURL, teamcenterInstance);
    // Check out the dataset with specific parameters for
editing in TIA Portal
    tcGatewayLockProvider.CheckoutDataset(connectionInfo
,"000495","A",
    DatasetType.T4TiaProjectDataset,"Project30");
    // Use TcGatewaySearchAndDownloadProvider to download the
dataset as a file for
    // local processing
    TcGatewaySearchAndDownloadProvider downloadFromTcProvider
= tiaPortal.GetService<TcGatewaySearchAndDownloadProvider>();
```

```
    FileInfo TIAObjectStarterFilePath =
downloadFromTcProvider.Download(connectionInfo,"000495", "A",
ItemType.Project, LocalCacheOption.Overwrite);
    var project =
tiaPortal.Projects.Open(TIAObjectStarterFilePath);
    //OR
    var library =
tiaPortal.GlobalLibraries.Open(TIAObjectStarterFilePath,
OpenMode.ReadWrite);
    //User can work on the Project.
    project.DeviceGroups.Create("Group_3");
    project.Save();
    }
    catch(TcGatewayException ex)
    {
    // other operations
    }
}
```

1.14 Searching Project / Global library from Teamcenter

Requirement

- The TIA Portal Openness is connected to the TIA Portal
  See Connecting to the TIA Portal
- A Project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to search project and global library in Teamcenter. The Search API returns a list of TIA Portal objects from Teamcenter with search result containing the details such as ItemID and List of related RevisionIds.

Parameter

Parameter	Data type	Is Mandatory	Description

tcGatewayConnectionInfo	Siemens.Engineering.TeamcenterGateway.TcGatewayConnectionInfo	Yes
type	Siemens.Engineering.TeamcenterGateway.ItemType(Enum)	Yes
TIAObjectName	System.String	No
itemID	System.String	No
itemName	System.String	No

			wild card characters. [User should provide value in at least one of the fields; Item ID, Item Name, TIAObjectName]
revisionID	System.String	No	Specifies the revision ID of the TIA Project / Library. User shall either enter a complete revision ID or with wild card characters.

Note

A Siemens.Engineering exception of type TcGatewayException is thrown if the required mandatory parameters are not passed to search project / library in Teamcenter.

Program code



```
Copies the following program code to clipboard.
//Program code: Search project/library in Teamcenter
private void SearchProjectInLibraryTeamCenter(SecureString password)
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    // Retrieve TcGatewayConnectionProvider from GetService()using tiaPortal instance
    TeamcenterConnectionProvider tcGatewayConnectionProvider = tiaPortal.GetService<TeamcenterConnectionProvider>();
    string userName = "username";
    string userRole = "userrole";
    string userGroup = "usergroup";
    string serverURL = "teamcenterserver_url";
    string teamcenterInstance = "teamcenterInstance";
    try
    {
    // Connection to the Teamcenter using user's credentails
```

```
    TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.Connect(userName, password,
userGroup, userRole, serverURL, teamcenterInstance);
    // Search for items of type 'Project' starting with 'P*'
    TcGatewaySearchAndDownloadProvider searchAndLoadProvider
= tiaPortal.GetService<TcGatewaySearchAndDownloadProvider>();
    var items = searchAndLoadProvider.Search(connectionInfo,
ItemType.Project, "P*", "", "", "");
    foreach (var item in items)
    {
    var itemId = item.ItemId;
    var revisionIdList = item.RevisionId;
    // Display information about each revision of the item
        foreach (var revision in revisionIdList)
        {
        Console.WriteLine(itemId + "-" + revision);
        }
    }
    }
    catch (TcGatewayException ex)
    {
     // other operations
    }
}
```

1.15 Checkout Project/Global library dataset in Teamcenter

Requirement

- The TIA Portal Openness is connected to the TIA Portal
  See Opening a project
- A Project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to perform checkout action on TIA Portal project and Global library in Teamcenter dataset.

Parameter

Parameter	Data type	Is Manc
tcGatewayConnectionInfo	Siemens.Engineering.TeamcenterGateway.TcGatewayConnectionInfo	Yes
itemID	System.String	Yes
revisionID	System.String	Yes
datasetType	Siemens.Engineering.TeamcenterGateway.DatasetType	Yes
datasetName	System.String	Yes

Note

A Siemens.Engineering exception of type TcGatewayException is thrown if the required mandatory parameters are not passed to perform checkout action on project / library in Teamcenter.

Program code: Checkout dataset in Teamcenter



```
Copies the following program code to clipboard.
//Program code: Checkout project/global library in Teamcenter dataset
private void CheckoutProjectInTeamcenterDataset(SecureString password)
{
    // Refer the Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    // Retrieve TcGatewayConnectionProvider and TcGatewayLockProvider
    // from GetService()using tiaPortal instance
    TeamcenterConnectionProvider tcGatewayConnectionProvider = tiaPortal.GetService<TeamcenterConnectionProvider>();
    TcGatewayLockProvider tcGatewayLockProvider = tiaPortal.GetService<TcGatewayLockProvider>();
    try
    {
    // Define user-specific credentials and connection details
    string userName = "username";
    string userRole = "userrole";
    string userGroup = "usergroup";
    string serverURL = "serverURL";
    string teamcenterInstance = "teamServerInstance";
    // Connect to the Teamcenter using credentials
    TcGatewayConnectionInfo connectionInfo = tcGatewayConnectionProvider.Connect(userName, password, userGroup, userRole, serverURL, teamcenterInstance);
```

```
    // Checkout the dataset in Teamcenter
    tcGatewayLockProvider.CheckoutDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");
    }
    catch (TcGatewayException)
    {
    //User can perform saving of Changes of TIA Project to
the checked out dataset
    // or any other operations
    }
}
```

1.16 Checkin Project/Global library dataset in Teamcenter

Requirement

- The TIA Portal Openness is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to check in action on TIA Portal project and Global library in Teamcenter dataset.

Parameter

Parameter name	Data type	Is Manc
tcGatewayConnectionInfo	Siemens.Engineering.TeamcenterGateway.TcGatewayConnectionInfo	Yes

			tcGatewayConnectionInfo .
itemID	System.String	Yes	Specifies the item ID of the TIA Portal Project / Library in Teamcenter
revisionID	System.String	Yes	Specifies the revision ID of the TIA Portal Project / Library in Teamcenter to which the dataset is linked.
datasetType	Siemens.Engineering.TeamcenterGateway.DatasetType	Yes	Specifies the Teamcenter dataset type corresponding to either project / library to be checked in. Eg: T4TiaProjectDataset / T4TiaLibraryDataset.
datasetName	System.String	Yes	Specifies the Teamcenter dataset name corresponding to either project / library to be checkedin.

Note

A Siemens.Engineering exception of type TcGatewayException is thrown if any of the above mandatory parameters is not passed for checkin of project / library to the Teamcenter dataset.

Program code

```
Copies the following program code to clipboard.
//Program code: Checkin project/global library in Teamcenter dataset
private void CheckinProject(SecureString password)
{
    // Refer Connecting to the TIA Portal section
```

```
    TiaPortal tiaPortal = new TiaPortal();
    {
    // Retrieve TcGatewayConnectionProvider and
TcGatewayLockProvider
    // from GetService() using tiaPortal instance
    TeamcenterConnectionProvider tcGatewayConnectionProvider
= tiaPortal.GetService<TeamcenterConnectionProvider>();
    TcGatewayLockProvider tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();
    try
    {
    // Define user specific credentials and connection
details
    string userName = "username";
    string userRole = "userole";
    string userGroup = "usergroup";
    string serverURL = "teamcenterserver_URL";
    string teamcenterInstance = "teamcenter_instance";
    // Connect to Teamcenter using provided credentials
    TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.Connect(userName, password,
    userGroup, userRole, serverURL, teamcenterInstance);
    // User can perform save changes of TIA Project to the
checkedout dataset
    // using TcGateway Save APIs.
    tcGatewayLockProvider.CheckoutDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");
    // Checkin to release the lock on dataset
    tcGatewayLockProvider.CheckinDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");
    }
    catch (TcGatewayException ex)
    {
```

```
     // other operations
    }
    }
}
```

1.17 Cancel checkout dataset in Teamcenter

Requirement

- The TIA Portal Openness is connected to the TIA Portal
  See Connecting to the TIA Portal
- A Project is open
  See Opening a project

Application

You can use the TIA Portal Openness to cancel checkout action on TIA Portal project and Global library dataset in Teamcenter. You can also use the CancelCheckoutDataset() to release the lock on dataset in Teamcenter.

Parameter

Parameter name	Data type	Is Manc
tcGatewayConnectionInfo	Siemens.Engineering.TeamcenterGateway.TcGatewayConnectionInfo	Yes
itemID	System.String	Yes
revisionID	System.String	Yes

			in Teamcenter to which the dataset is linked.
datasetType	Siemens.Engineering.TeamcenterGateway.DatasetType	Yes	Specifies the Teamcenter dataset type corresponding to either project / library to be checked out. Eg: T4TiaProjectDataset / T4TiaLibraryDataset.
datasetName	System.String	Yes	Specifies the Teamcenter dataset name corresponding to either project / library to be checked out.

Note

A Siemens.Engineering exception of type TcGatewayException is thrown if any of the above mandatory parameters is not passed to cancel checkout action on project / library in Teamcenter.

Program code

```
Copies the following program code to clipboard.
//Program code: Cancel checkout datatset in Teamcenter
private void CancelCheckoutDataset(SecureString password)
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    // Retrieve the TcGatewayConnectionProvider and
TcGatewayLockProvider
    // from GetService() using tiaPortal instance
```

```
    TeamcenterConnectionProvider tcGatewayConnectionProvider =
tiaPortal.GetService<TeamcenterConnectionProvider>();
    TcGatewayLockProvider tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();
    try
    {
    // Define user specific credentials and connection details
    string userName = "your_username";
    string userRole = "your_userrole";
    string userGroup = "your_usergroup";//Group
    string serverURL = "teamcenterserverURL";
    string teamcenterInstance = "teamcenterInstance";
    // Connect to Teamcenter using provided credentials
    TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.Connect(userName, password,
    userGroup, userRole, serverURL, teamcenterInstance);
    //User can perform saving of Changes of TIA Project to the
checked out dataset
    //using TcGateway Save APIs.
    tcGatewayLockProvider.CheckoutDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");
    //Cancel checkout on dataset to release the lock on dataset
and undo the changes
    //done on dataset
    tcGatewayLockProvider.CancelCheckoutDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");
    }
    catch (TcGatewayException ex)
    {
    // other operations
    }
}
```

Functions for Connections

2

2.1 Configurable attributes of a port-to-port connection

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Opening a project
- A project is open.

  See Opening a project

Application

The attributes of a port interconnection are located at the port device item. The read and write access of attributes via TIA Portal Openness is the same as at the UI.

Port interface settings

The following attributes are provided for port interface settings:

Attribute name	Data type	Writeable	Access	Description
`MediumAttachmentType`	MediumAttachmentType	r/o	Dynamic attribute	-
`CableName`	CableName	r/w	Dynamic attribute	-
`AlternativePartnerPorts`	Boolean	r/w	Dynamic attribute	Only available if toolchanger functionality is supported, e.g. at CPU1516.
`SignalDelaySelection`	SignalDelaySelection	r/w	Dynamic attribute	-

CableLength	CableLength	r/w	Dynamic attribute	-
SignalDelayTime	Double	r/w	Dynamic attribute	-

The following ENUM values are provided for the attribute MediumAttachmentType:

Value	Description
MediumAttachmentType.None	Attachment type cannot be determined.
MediumAttachmentType.Copper	Attachment type is copper.
MediumAttachmentType.FibreOptic	Attachment type is fiber optic.

The following ENUM values are provided for the attribute CableName:

Value	Description
CableName.None	No cable name is specified
CableName.FO_Standard_Cable_9	FO standard cable GP (9 µm)
CableName.Flexible_FO_Cable_9	Flexible FO cable (9 µm)
CableName.FO_Standard_Cable_GP_50	FO standard cable GP (50 µm)
CableName.FO_Trailing_Cable_GP	FO trailing cable / GP
CableName.FO_Ground_Cable	FO ground cable
CableName.FO_Standard_Cable_62_5	FO standard cable (62.5 µm)
CableName.Flexible_FO_Cable_62_5	Flexible FO cable (62.5 µm)
CableName.POF_Standard_Cable_GP	POF standard cable GP
CableName.POF_Trailing_Cable	POF trailing cable
CableName.PCF_Standard_Cable_GP	PCF standard cable GP
CableName.PCF_Trailing_Cable_GP	PCF trailing cable / GP
CableName.GI_POF_Standard_Cable	GI-POF standard cable

CableName.GI_POF_Trailing_Cable	GI-POF trailing cable
CableName.GI_PCF_Standard_Cable	GI-PCF standard cable
CableName.GI_PCF_Trailing_Cable	GI-PCF trailing cable

The following ENUM values are provided for the attribute SignalDelaySelection:

Value	Description
SignalDelaySelection.None	-
SignalDelaySelection.CableLength	CableLength is used to define the signal delay.
SignalDelaySelection.SignalDelayTime	SignalDelayTime is used to define the signal delay.

The following ENUM values are provided for the attribute CableLength:

Value	Description
CableLength.None	Cable length is not specified.
CableLength.Length20m	Cable length is 20m.
CableLength.Length50m	Cable length is 50m.
CableLength.Length100m	Cable length is 100m.
CableLength.Length1000m	Cable length is 1000m.
CableLength.Length3000m	Cable length is 3000m.

Port options

The following attributes are provided for port options:

Attribute name	Data type	Writeable	Access
PortActivation	bool	r/w	Dynamic attribute

TransmissionRateAndDuplex	TransmissionRateAndDuplex	r/w	Dynamic attribute
PortMonitoring	bool	r/w	Dynamic attribute
TransmissionRateAutoNegotiation	bool	r/w	Dynamic attribute
EndOfDetectionOfAccessibleDevices	bool	r/w	Dynamic attribute
EndOfTopologyDiscovery	bool	r/w	Dynamic attribute
EndOfSyncDomain	bool	r/w	Dynamic attribute

The following ENUM values are provided for the attribute TransmissionRateAndDuplex:

Value	Description
TransmissionRateAndDuplex.None	-
TransmissionRateAndDuplex.Automatic	Automatic
TransmissionRateAndDuplex.AUI10Mbps	10 Mbps AUI
TransmissionRateAndDuplex.TP10MbpsHalfDuplex	TP 10 Mbps half duplex
TransmissionRateAndDuplex.TP10MbpsFullDuplex	TP 10 Mbps full duplex
TransmissionRateAndDuplex.AsyncFiber10MbpsHalfDuplex	async fiber 10Mbit/s half duplex mode
TransmissionRateAndDuplex.AsyncFiber10MbpsFullDuplex	async fiber 10Mbit/s full duplex mode
TransmissionRateAndDuplex.TP100MbpsHalfDuplex	TP 100 Mbps half duplex

`TransmissionRateAndDuplex.TP100MbpsFullDuplex`	TP 100 Mbps full duplex
`TransmissionRateAndDuplex.FO100MbpsFullDuplex`	FO 100 Mbps full duplex
`TransmissionRateAndDuplex.X1000MbpsFullDuplex`	X1000 Mbps full Duplex
`TransmissionRateAndDuplex.FO1000MbpsFullDuplexLD`	FO 1000 Mbps full duplex LD
`TransmissionRateAndDuplex.FO1000MbpsFullDuplex`	FO 1000 Mbps full Duplex
`TransmissionRateAndDuplex.TP1000MbpsFullDuplex`	TP 1000 Mbps full duplex
`TransmissionRateAndDuplex.FO10000MbpsFullDuplex`	FO 10000 Mbps full Duplex
`TransmissionRateAndDuplex.FO100MbpsFullDuplexLD`	FO 100 Mbps full duplex LD
`TransmissionRateAndDuplex.POFPCF100MbpsFullDuplex`	POF/PCF 100 Mbps full duplex

See also

Connecting to the TIA Portal

Functions on libraries

3

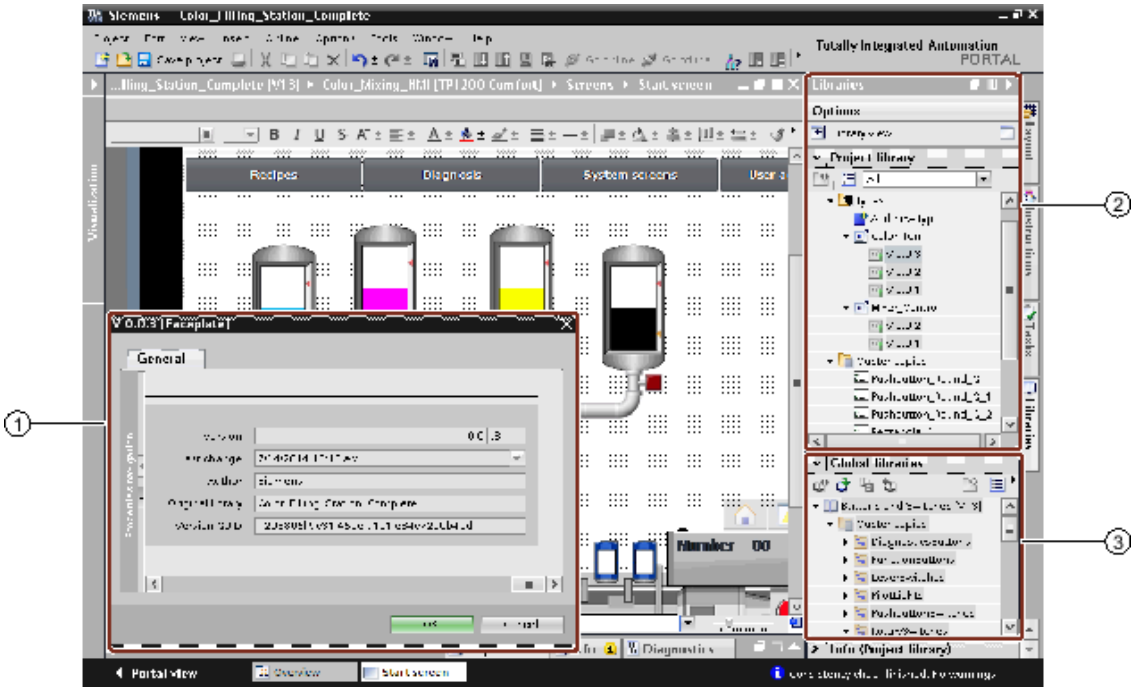
3.1 Functions for objects and instances

Accessing types and instances

You can use the TIA Portal Openness API interface to access types, type versions and master copies in the project library or global libraries. You can determine connections between type versions and instances. You can also update instances in the project and synchronize changes between a global library and the project library. The TIA Portal Openness API interface also supports the comparison of types versions and instances.

Functions for objects and instances

You have access to the following functions for types, type versions, master copies and instances with the TIA Portal Openness API interface:



① Display attributes of types, type versions, master copies and instances

② The following functions are available in the project library:

- Update instances of types
- Instantiating type versions in the project
- Navigate within the library group
- Delete groups, types, type versions and master copies

③ The following functions are available in the global library:

- Update instances of types
- Instantiate type version in the project
- Navigate within the library group

3.2 Accessing global libraries

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal

Application

Three types of Global Libraries are existing.

- System global library (Siemens.Engineering.Library.SystemGlobalLibrary): These global libraries are included as part of a TIA Portal installation and use the .asx file extension. All system global libraries are read only.
- Corporate global library (Siemens.Engineering.Library.CorporateGlobalLibrary): These global libraries have been chosen by an administrator to be preloaded when TIA Portal is started. All corporate global libraries are read only.
- User global library (Siemens.Engineering.Library.UserGlobalLibrary): These global libraries have been created by users of TIA Portal. User global libraries can be opened either as read only mode or readwrite mode.
  If an user global library is already openned in a certain mode then the same user global library cannot be openned with another mode.
  User global libraries from previous versions can be opened only in read only mode.

Global Libraries opened using TIA Portal Openness will also be added to the TIA Portal UI's global library collection, and seen in the TIA Portal UI if the UI is present.

Attributes

Attributes of GlobalLibrary:

Attribute	Data type	Description	Access
Author	string	Specifies the author of the global library.	Read
Name	string	Specifies the name of the global library.	Read

IsModified	boolean	True if the contents of the global library has been modified.	Read
IsReadOnly	boolean	Specifies whether the global library is open to read only or not.	Read
Path	FileInfo	Specifies the path of the global library.	Read
LanguageSettings	LanguageSettings	Specifies the global library language settings	Read
Comment	MultilingualText	Specifies the comment of the global library.	Read
IsWriteProtected[1]	boolean	Specifies whether the global library is write protected or not	Read
LastModifiedBy[2]	string	Specifies the user who modified	

		the global library last time	Read
Size[3]	long	Specifies the size of the global library in KB	Read
CreationTime[4]	DateTime	Specifies the time at which the library was created	Read
LastModified[5]	DateTime	Specifies the last time at which the global library was modified	Read
Version[6]	string	Specifies the version of the global library	Read
Family[7]	string	Specifies the global library family	Read
Copyright[8]	string	Specifies the global library copyright information	Read
HistoryEntries[9]	Siemens.Engineering.HistoryEntryComposition	Specifies the log of global library	

		history events	Read
UsedProducts[10]	Siemens.Engineering.UsedProductComposition	Specifies the composition of used products in the global library	Read

Note

The attribute name mentioned above with superscript is supported starting from TIA Portal Openness v20.

Attributes of GlobalLibraryInfo:

Attribute name	Return type	Description
IsReadOnly	boolean	True if the global library is read only.
IsOpen	boolean	True if the global library is already open.
LibraryType	GlobalLibraryType	Type of the global library: • System: System global library • Corporate: Corporate global library • User: User global library
Name	string	Specifies the name of the global library.
Path	FileInfo	Specifies the path of the global library.

Program code: Access properties of global library

Modify the following program code to access the properties of a global library :

```
Copies the following program code to clipboard.

private static void
GetLibraryProperties(Siemens.Engineering.Library.GlobalLibrary
globalLibrary)

{
```

```
    string author = globalLibrary.Author;
    string name = globalLibrary.Name;
    bool isModified = globalLibrary.IsModified;
    bool isReadOnly = globalLibrary.IsReadOnly;
    bool isWriteProtected = globalLibrary.IsWriteProtected;
    string lastModifiedBy = globalLibrary.LastModifiedBy;
    long size = globalLibrary.Size;
    FileInfo path = globalLibrary.Path;
    DateTime creationTime = globalLibrary.CreationTime;
    DateTime lastModified = globalLibrary.LastModified;
    string version = globalLibrary.Version;
    string family = globalLibrary.Family;
    string copyright = globalLibrary.Copyright;
    Siemens.Engineering.HistoryEntryComposition
historyEntries = globalLibrary.HistoryEntries;
    Siemens.Engineering.LanguageSettings languageSettings =
globalLibrary.LanguageSettings;
    Siemens.Engineering.MultilingualText comment =
globalLibrary.Comment;
    Siemens.Engineering.UsedProductComposition usedProducts =
globalLibrary.UsedProducts;
}
```

Program code: Available global libraries

Modify the following program code to get informations about all available global libraries:

```
Copies the following program code to clipboard.
TiaPortal tiaPortal = ...;
var availableLibraries =
tia.GlobalLibraries.GetGlobalLibraryInfos();
foreach (GlobalLibraryInfo info in availableLibraries)
{
Console.WriteLine(" Library Name: {0}", info.Name);
```

```
Console.WriteLine(" Library Path: {0}", info.Path);
Console.WriteLine(" Library Type: {0}", info.LibraryType);
Console.WriteLine(" Library IsOpen: {0}", info.IsOpen);
}
```

See also

Accessing folders in a library

3.3 Accessing library base type

Requirement

- The application is connected to the TIA Portal using TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to access the base type classes of the library type object. These all have a direct base type of Siemens.Engineering.Library.Types.LibraryType

You can have access to the below base type classes:

- Siemens.Engineering.Hmi.Faceplate.FaceplateLibraryType
- Siemens.Engineering.Hmi.RuntimeScripting.VBScriptLibraryType
- Siemens.Engineering.Hmi.RuntimeScripting.CScriptLibraryType
- Siemens.Engineering.Hmi.Screen.ScreenLibraryType
- Siemens.Engineering.Hmi.Screen.StyleLibraryType
- Siemens.Engineering.Hmi.Screen.StyleSheetLibraryType
- Siemens.Engineering.Hmi.Tag.HmiUdtLibraryType
- Siemens.Engineering.SW.Blocks.CodeBlockLibraryType
- Siemens.Engineering.SW.Types.PlcTypeLibraryType

Program code

```
Copies the following program code to clipboard.
TiaPortal tiaPortal = ...;
Project project = tia.Projects.First();
```

```
ProjectLibrary library = project.ProjectLibrary;
//Pre-V14SP1
LibraryType libraryType=
library.TypeFolder.Types.Find("VBFunction_1");
//V14SP1
VBScriptLibraryType libraryTypeAsVbScript = libraryType as
VBScriptLibraryType;
```

3.4 Accessing global library languages

Requirements

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A library is open
  See Opening libraries

Application

You can use language setting navigator to access and manage the Global Library languages.

TIA Portal Openness supports the following access to the global library languages:

- Iterating through supported languages.
- Searching through the collection of supported languages by `System.Globalization.CultureInfo`.
- Accessing individual languages. Each Language object will contain a single read-only attribute Culture of type `System.Globalization.CultureInfo`.
- Accessing a collection of active languages.
- Searching through the collection of active languages by `System.Globalization.CultureInfo`.
- Adding a language to a collection of active languages.
- Removing a language from a collection of active languages.
- Setting an editing language.
- Setting a reference language.

Attribute of global library languages

Global library languages provides the following attribute:

Attribute name	Data type	Writeable	Description
LanguageSettings	Siemens.Engineering.LanguageSettings	r/o	Handles global library languages

Program code: Enumerating global library language

Modify the following program code to enumerate the global library language:

```
Copies the following program code to clipboard.

FileInfo m_GlobalLibrarypath = new FileInfo("bla");
TiaPortal portal = new
TiaPortal(TiaPortalMode.WithUserInterface);
var globalLibrary =
portal.GlobalLibraries.Open(m_GlobalLibrarypath,
OpenMode.ReadOnly);
LanguageSettings languageSettings =
globalLibrary.LanguageSettings;
```

Program code: Accessing language setting

Modify the following program code to access the language setting on global library:

```
Copies the following program code to clipboard.
```

```
FileInfo m_GlobalLibrarypath = new FileInfo("bla");
TiaPortal portal = new
TiaPortal(TiaPortalMode.WithUserInterface);
var globalLibrary =
portal.GlobalLibraries.Open(m_GlobalLibrarypath,
OpenMode.ReadOnly);
LanguageComposition languages =
globalLibrary.LanguageSettings.Languages;
foreach (Language language in languages)
{
// Work with this language
}
```

Program code: Setting global library languages

Modify the following program code to set global library languages. If you set a new supported language through TIA Portal Openness, the language will be added to the active languages collection.



```
Copies the following program code to clipboard.
FileInfo m_GlobalLibrarypath = new FileInfo("bla");
TiaPortal portal = new
TiaPortal(TiaPortalMode.WithUserInterface);
var globalLibrary =
portal.GlobalLibraries.Open(m_GlobalLibrarypath,
OpenMode.ReadOnly);
LanguageSettings languageSettings =
globalLibrary.LanguageSettings;
LanguageComposition supportedLanguages =
languageSettings.Languages;
LanguageAssociation activeLanguages =
languageSettings.ActiveLanguages;
Language supportedGermanLanguage =
supportedLanguages.Find(CultureInfo.GetCultureInfo("de-DE"));
activeLanguages.Add(supportedGermanLanguage);
languageSettings.EditingLanguage = supportedGermanLanguage;
```

```
languageSettings.ReferenceLanguage = supportedGermanLanguage;
```

Note

Adding and modifying languages in global library language setting does not modify languages of released versions in the global library. This behaviour holds true for updating global library languages through UI (language editor) or LanguageSettings navigator.

3.5 Opening libraries

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application. This requirement is only for accessing project libraries.

  See Opening a project

Application

A global library can be opened using System.IO.FileInfo with a path to the library file on a local storage medium or a network storage. Only user global libraries ca be opened by path. A path obtained from a system global library or a corporate global library can not be used to open it.

As of V14 SP1 global libraries can be opened using the GlobalLibraryInfo. The OpenMode is specified in the GlobalLibraryInfo.

A user global library from a previous version of TIA Portal can be upgraded and opened with the current version of TIA Portal. A global library from V13 or a previous version cannot be opened with upgrade. These libraries have to be upgraded to V13 SP1 first.

Libraries opened using TIA Portal Openness will also be added to the global library collection in the TIA Portal, and will be visible in the user interface of TIA Portal.

Program code: Opening a library using System.IO.FileInfo

Modify the following program code:

```
 Copies the following program code to clipboard.

TiaPortal tia = ...
FileInfo fileInfo = ....
UserGlobalLibrary userLib =
tia.GlobalLibraries.Open(fileInfo, OpenMode.ReadWrite);
```

Program code: Opening a library using GlobalLibraryInfo

Modify the following program code:

```
Copies the following program code to clipboard.

TiaPortal tia = ...
IList<GlobalLibraryInfo> libraryInfos =
tia.GlobalLibraries.GetGlobalLibraryInfos();
GlobalLibraryInfo libInfo = ...; //check for the info you
need from the list, e.g.
GlobalLibrary libraryOpenedWithInfo;
if (libInfo.Name == "myLibrary")
libraryOpenedWithInfo = tia.GlobalLibraries.Open(libInfo);
```

Program code: Upgrading a library

Modify the following program code:

```
Copies the following program code to clipboard.

TiaPortal tia = ...
FileInfo fileInfo = .... //library from previous TIA Portal
version
UserGlobalLibrary userLib =
tia.GlobalLibraries.OpenWithUpgrade(fileInfo);
```

OpenMode

Value	Description
ReadOnly	Read access to the library.
ReadWrite	Read and write access to the library.

3.6 Comparing libraries

Requirement

- The application is connected to the TIA Portal using TIA Portal Openness
  See Connecting to the TIA Portal

- The project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to compare the libraries with each other. For example, you can use the feature to find the differences between versions before updating the new version into the project.

You can use the Openness to do the detailed comparision of selected library objects. It can compare two library objects

- Type object
- Version object
- Master copies

The Openness API allows you to compare two type objects for example version object as well as master copies and returns the detailed content compare result.

Method, Parameter and Enum

The TIA Portal Openness supports the following method with its signature type to compare the library objects :

Method	Return type
DetailedCompareResult CompareTo(LibraryType targetLibraryObject)	Siemens.Engineering.Library.Types.LibraryType
DetailedCompareResult CompareTo(LibraryTypeVersion targetLibraryObject)	Siemens.Engineering.Library.Types.LibraryTypeVersion
DetailedCompareResult CompareTo(MasterCopy targetLibraryObject)	Siemens.Engineering.Library.MasterCopies.MasterCopy

The LibraryCompareResult supports the following navigator in TIA Portal Openness:

Name	Types
Parent	Siemens.Engineering.IEngineeringObject
RootElement	Siemens.Engineering.Library.Compare.LibraryCompareResultElement

The LibraryCompareResultElement supports the following navigator and attribute name in TIA Portal Openness:

Name	Types
Parent	Siemens.Engineering.Library.Compare.LibraryCompareResultElementComposition
RootElement	Siemens.Engineering.Library.Compare.LibraryCompareResultElement

Attribute name	Type
ComparisionResult	Siemens.Engineering.Library.Compare.LibraryCompareResultState
DetailedInformation	System.String
LeftName	System.String
RightName	System.String
Left	Siemens.Engineering.IEngineeringObject
Right	Siemens.Engineering.IEngineeringObject

The LibraryCompareResultState ENUM supports the following ENUM items:

Enum Items	Description
ContainerContentsDifferent	Container contents has one or more differences
ContainerContentsIdentical	Container content is identical
ObjectsDifferent	Objects are different
LeftMissing	Left object is missing
RightMissing	Right object is missing
ObjectsIdentical	Objects are identical

The DetailedCompareResult supports the following navigator in TIA Portal Openness:

Name	Type

Parent	Siemens.Engineering.IEngineeringObject
RootElement	Siemens.Engineering.Library.Compare.LibraryCompareResultElement

The DetailedCompareResultElement supports the following navigator and attribute name in TIA Portal Openness:

Name	Type
Parents	Siemens.Engineering.IEngineeringObject

Attribute name	Type	Description
Description	System.String	Detail compare property name
LeftValue	System.Object	Detail compare left property value. The LeftValue property supports the following types, which are System.Guid, System.Boolean, System.String, System.DateTime
RightValue	System.Object	Detail compare right property value. The RightValue property supports the following types, which are System.Guid, System.Boolean, System.String, System.DateTime.
DetailCompareStatus	Siemens.Engineering.Library.Compare.DetailCompareStatus	Returns the comparison result of the property values based on

		the default criteria filter applied. (Original library property of version object is not compared by default)

The DetailCompareStatus supports the following ENUM items:

Enum Items	Value
ObjectsDifferent	Objects are different
ObjectsIdentical	Objects are identical
NotCompared	When filters are applied, objects are not compared

Program code

```
Copies the following program code to clipboard.
...
GlobalLibrary globalLibrary = ...;
ProjectLibrary projectLibrary = Project.ProjectLibrary;
LibraryCompareResult libraryCompareResults =
globalLibrary.CompareToLibrary(projectLibrary);
...
```

Modify the following program code to compare two type objects:

```
Copies the following program code to clipboard.
...
GlobalLibrary globalLibrary = ...;
ProjectLibrary projectLibrary = Project.ProjectLibrary;
LibraryType leftTypeObject =
ProjectLib.TypeFolder.Types.Find(typeName);
LibraryType rightTypeObject =
GlobalLib.TypeFolder.Types.Find(typeName);
DetailedCompareResult detailedCompareResult =
leftTypeObject.CompareTo(rightTypeObject);
...
```

Modify the following program code to compare two versions:

```
Copies the following program code to clipboard.

...
GlobalLibrary globalLibrary = ...;
ProjectLibrary projectLibrary = Project.ProjectLibrary;
LibraryTypeVersion leftVersionObject =
ProjectLib.TypeFolder.Types.Find(typeName).Versions[0];
LibraryTypeVersion rightVersionObject =
GlobalLib.TypeFolder.Types.Find(typeName).Versions[0];
DetailedCompareResult detailedCompareResult =
leftVersionObject.CompareTo(rightVersionObject);
...
```

Modify the following program code to compare two master copy objects:

```
Copies the following program code to clipboard.

...
GlobalLibrary globalLibrary = ...;
ProjectLibrary projectLibrary = Project.ProjectLibrary;
LibraryTypeVersion leftMasterCopy =
ProjectLib.MasterCopyFolder.MasterCopies.Find(masterCopyName);
LibraryTypeVersion rightMasterCopy =
GlobalLib.MasterCopyFolder.MasterCopies.Find(masterCopyName);
DetailedCompareResult detailedCompareResult =
leftMasterCopy.CompareTo(rightMasterCopy);
...
```

3.7 Enumerating open libraries

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal

Application

All opened global libraries in the TIA Portal, regardless if they have been opened via API or via user interface can be enumerated.

Global Libraries from previous versions of TIA Portal will not be enumerated if they are opened with write access.

Program code

Modify the following program code to enumerate open global libraries.

```
Copies the following program code to clipboard.
TiaPortal tia = ...
foreach (GlobalLibrary globLib in tia.GlobalLibraries)
{
    ////work with the global library
}
```

See also

Opening a project

3.8 Saving and closing libraries

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A library is open.

  See Opening libraries

Application

User global libraries can be closed or saved. Any changes made to the global library will not be saved automatically. All unsaved changes will be discarded without prompting by closing a global library.

System global libraries and corporate global library cannot be closed or saved.

To save and close a global library:

- Use the Save ( ) method to save a user global library
- Use the SaveAs ( ) method to save a user global library in a different directory
- Use the Close ( ) method to close a user global library

Feature tokens included for SaveAs ( ) are:

- PublicAPI : Necessary for any Published feature

- DenyIfTransaction : Necessary since SaveAs should not be allowed within a transaction as it cannot be undone.

Program code

Modify the following program code to save a user global library:

```
Copies the following program code to clipboard.
UserGlobalLibrary userLib = ...
// save changes and close library
userLib.Save();
userLib.Close();
```

Modify the following program code to save a user global library in a different location:

```
Copies the following program code to clipboard.
TiaPortal portal = new TiaPortal(TiaPortalMode.WithUserInterface);
GlobalLibraryComposition globalLibraryComposition =
portal.GlobalLibraries;
//please adapt the path and the extension alx to the installed
version of TIA Portal
FileInfo existingLibraryfileInfo = new
FileInfo(@"D:\GlobalLibraries\MyGlobalLibrary\MyGlobalLibrary.alx");
DirectoryInfo targetDirectoryInfo = new
DirectoryInfo(@"D:\GlobalLibraries\GlobalLibrarySaveAs");
UserGlobalLibrary userGlobalLibary =
globalLibraryComposition.Open(existingLibraryfileInfo,
OpenMode.ReadWrite);
userGlobalLibary.SaveAs(targetDirectoryInfo);
```

Note

The persistence of the library is changed after SaveAs operation. Thus, after the SaveAs is performed, the newly saved library is available in libraries card. However, the original library on which SaveAs operation was performed, would not be available. The newly saved library would have the same mode as the original library on which SaveAs was performed. This means if the existing library was opened in ReadOnly mode, then the newly saved library in the target path would also open in the ReadOnly mode. Similar is the case for ReadWrite mode.

Modify the following program code to close a user global library:

```
Copies the following program code to clipboard.
UserGlobalLibrary userLib = ...
// close and discard changes
userLib.Close();
```

3.9 Archiving and retrieving a library

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  SeeConnecting to the TIA Portal
- A library is open.
  See Opening libraries
- A library is save
  See Saving and closing libraries

Introduction

You can use the TIA Portal Openness to archive the open and save global libraries prior to any further modification to prevent from unintended result so that you canuse the TIA Portal Openenss to retrieve the archived library. You can also share the archived file across the network easily.

Archiving a library

You can use the TIA Portal Openness API interface to archive a user global library. The API is available on the Siemens.Engineering.UserGlobalLibrary object.

The API definition looks like the following:

```
Copies the following program code to clipboard.
public void Archive(System.IO.DirectoryInfo targetDirectory,
string targetName,
Siemens.Engineering.Library.LibraryArchivationMode
archivationMode)
```

The targetName is the name of the file created for archived or non archived. This file may or may not contain any file extensions. If you do not provide any extension or provide an extension apart from "zalx" or "zal14" etc., then the archived file could not be retrieved from TIA Portal outside the Openness API.
For LibraryArchivationMode value as Compressed and DiscardRestorableDataAndCompressed,

the archived file name is same as it is provided by you. For LibraryArchivationMode value as None and DiscardRestorableData, the Library file extension is automatically decided by TIA Portal Project Manager component, based on the current version of TIA Portal.

Note

You must have saved the library before calling Archive API. In case the library contains any unsaved changes, archive will throw an EngineeringTargetInvocationException.

Library Archivation Mode

The LibraryArchivationMode enumeration have four values:

LibraryArchivationMode	Description
None	• No special action are taken with the orginial files. Mode is similiar to a "save as" operation. • No compressed zip file is created in this mode. • The difference with SaveAs in this case is that the Archive will not change the persistence location to the new Archived folder whereas SaveAs does that.
DiscardRestorableData	• The file storage stores the library in an internal data file and this file grows whenever a library data modification happens. In the case of DiscardRestorableData mode this data file is reorganized (only latest version of the objects are stored and history is removed from the file) and Intermediate data, the files of the IM directory and tmp directory (see Library Directory structure) are not copied to the archive location. • No compressed zip file is created in this mode.
Compressed	The tmp library folder structure, created by Archiving, is compressed into a zip compatible archive. The tmp folder structure is removed after creation of the zip file.

DiscardRestorableDataAndCompressed	The tmp library folder structure, created by Archiving, discards the restorable data and then compressed into a zip compatible archive. The tmp folder structure is removed after creation of the zip file.

Program code: Archiving a library

Modify the following program code to archive a user global library:

```
var tiaPortal = new
TiaPortal(TiaPortalMode.WithoutUserInterface);

//Please adapt the path and the extension alx to the
installed version of TIA Portal

var libraryFilePath = @"E:\Sample1\Sample1.alx";

var userGlobalLibrary = tiaPortal.GlobalLibraries.Open(new
FileInfo(LibraryFilePath), OpenMode.ReadWrite);

var archivePath = @"E:\Archive";

var archiveFileName = "SampleArchive";

userGlobalLibrary.Archive(new DirectoryInfo(archivePath),
archiveFileName, LibraryArchivationMode.Compressed);
```

Retrieving a library

You can use the TIA Portal Openness API interface to retrieve an archived TIA Portal library. You can only retrieve a compressed archive. The API is available on the

Siemens.Engineering.GlobalLibraryComposition object.

The API definition looks like the following:

```
public Siemens.Engineering.Library.UserGlobalLibrary
Retrieve(System.IO.FileInfo sourcePath,
System.IO.DirectoryInfo targetDirectory,
Siemens.Engineering.OpenMode openMode)
```

Note

You cannot retrieve an archived library with 'LibraryArchivationMode.None' or 'LibraryArchivationMode.DiscardRestorableData' enumeration value.

You can call RetrieveWithUpgrade API for the archived library of a previous TIA Portal version. The API definition looks like the following:



```
Copies the following program code to clipboard.

public Siemens.Engineering.Library.UserGlobalLibrary
RetrieveWithUpgrade(System.IO.FileInfo sourcePath,
System.IO.DirectoryInfo

targetDirectory, Siemens.Engineering.OpenMode openMode)
```

Open Mode

The OpenMode enumeration have two values:

OpenMode	Description
ReadMode	Read access to the library. Data can be read from the library.
ReadWrite	Write access to the library. Data can be written to the library.

Program code: Retrieving a library

Modify the following program code to access to retrieve a library:



```
Copies the following program code to clipboard.

//Please adapt the path and the extension zalx to the
installed version of TIA Portal

var archivePath = @"E:\Archive\Sample1.zalx";

var retrievedLibraryDirectory = @"E:\RetrievedLibraries";

var tiaPortal = new
TiaPortal(TiaPortalMode.WithoutUserInterface);

tiaPortal.GlobalLibraries.Retrieve(new FileInfo(archivePath),
new DirectoryInfo(retrievedLibraryDirectory),
OpenMode.ReadWrite));
```

See also

Opening libraries

3.10 Creating global libraries

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal

Introduction

Global libraries can be created via TIA Portal Openness API by calling the Create method on the GlobalLibraryComposition. A UserGlobalLibrary will be returned

Parameters for creating global libraries

Parameter	Data Type	Type	Description
Author	String	Mandatory	Author of a global library.
Comment	String	Optional	Comment of a global library.
Name	String	Optional	Name of a global library
TargetDirectory	DirectoryInfo	Mandatory	Directory that will contain global library folder.

GlobalLibraryComposition.Create

Modify the following program code:



```
Copies the following program code to clipboard.

TiaPortal tia= ...;
DirectoryInfo targetDirectory = new
DirectoryInfo(@"D:\GlobalLibraries");
UserGlobalLibrary globalLibrary =
tia.GlobalLibraries.Create<UserGlobalLibrary>
(targetDirectory, "Library1")
```

According to this example:

- a folder "D:\GlobalLibraries\Library1" will be created
- a global library file "D:\GlobalLibraries\Library1\Library1.alXX" will be created

See also

3.11 Accessing folders in a library

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project
- You have access to the required library.
  See Accessing global libraries

Introduction

You can use the TIA Portal Openness API interface to access the system folders for types and master copies in a library. You can access types, type versions, master copies and user-defined folders within the system folder.

You can access a user-defined folder at any time using the `Find` method, for example `libTypeUserFolder.Folders.Find("SomeUserFolder");`.

Program code: Accessing system folders

Modify the following program code to access the system folder for types in a library:

```
 Copies the following program code to clipboard.
public static void AccessTypeSystemFolder(ILibrary library)
{
    LibraryTypeSystemFolder libTypeSystemFolder =
library.TypeFolder;
}
```

Modify the following program code to access the system folder for master copies in a library:

```
 Copies the following program code to clipboard.
public static void AccessMasterCopySystemFolder(ILibrary
library)
{
    MasterCopySystemFolder libMasterCopySystemFolder =
library.MasterCopyFolder;
```

```
}
```

Program code: Accessing user-defined folders via Find() method

```
Copies the following program code to clipboard.

...
LibraryTypeUserFolderComposition userFolderComposition = ...
LibraryTypeUserFolder userFolder =
userFolderComposition.Find("Name of user folder");
...
```

Program code: Enumerating user defined folders

Modify the following program code to enumerate user-defined subfolders in a system folder for types:

```
Copies the following program code to clipboard.

public static void
EnumerateUserFoldersInTypeSystemFolder(ILibrary library)
{
    // Enumerating user folders in type system folder:
    LibraryTypeSystemFolder libTypeSystemFolder =
library.TypeFolder;
    foreach (LibraryTypeUserFolder libTypeUserFolder in
libTypeSystemFolder.Folders)
    {
        //...
    }
}
```

Modify the following program code to enumerate user-defined subfolders in a system folder for master copies:

```
Copies the following program code to clipboard.

public static void
EnumerateUserFoldersInMasterCopySystemFolder(ILibrary
library)
```

```
{
    // Enumerating user folders in master copy system folder:
    MasterCopySystemFolder libMasterCopySystemFolder =
library.MasterCopyFolder;
    foreach (MasterCopyUserFolder libMasterCopyUserFolder in
libMasterCopySystemFolder.Folders)
    {
        //..
    }
}
```

Modify the following program code to enumerate user-defined subfolders in a user-defined folder for types:

```
Copies the following program code to clipboard.
public static void
EnumerateAllUserFolders(LibraryTypeUserFolder libUserFolder)
{
    foreach (LibraryTypeUserFolder libSubUserFolder in
libUserFolder.Folders)
    {
       EnumerateAllUserFolders(libSubUserFolder);
    }
}
```

Modify the following program code to enumerate user-defined subfolders in a user-defined folder for master copies:

```
Copies the following program code to clipboard.
public static void
EnumerateAllUserFolders(MasterCopyUserFolder libUserFolder)
{
    foreach (MasterCopyUserFolder libSubUserFolder in
libUserFolder.Folders)
    {
       EnumerateAllUserFolders(libSubUserFolder);
```

```
    }
}
```

Program code: Creating user-defined folders

Modify the following program code to create a user-defined folder for types:

```
Copies the following program code to clipboard.

var typeFolderComposition =
ProjectLibrary.TypeFolder.Folders;

var newTypeUserFolder =
typeFolderComposition.Create("NewTypeUserFolder");
```

Modify the following program code to create a user-defined folder for master copies:

```
Copies the following program code to clipboard.

var masterCopyFolderComposition =
projectProjectLibrary.MasterCopyFolder.Folders;

MasterCopyUserFolder newMasterCopyUserFolder =
masterCopyFolderComposition.Create("NewMasterCopyUserFolder);
```

Program code: Renaming user-defined folders

Modify the following program code to create a user-defined folder for types:

```
Copies the following program code to clipboard.

var typeUserFolder =
project.ProjectLibrary.TypeFolder.Folders.Find("SampleTypeUserFolderName

typeUserFolder.Name = "NewTypeUserFolderName";
```

```
Copies the following program code to clipboard.

var typeUserFolder =
ProjectLibrary.TypeFolder.Folders.Find("SampleTypeUserFolderName");

typeUserFolder.SetAttributes(new[] {new KeyValuePair<string,object>
("Name", "NewTypeUserFolderName")});
```

Modify the following program code to create a user-defined folder for master copies:

```
Copies the following program code to clipboard.
```

```
var masterCopyUserFolder =
project.ProjectLibrary.MasterCopyFolder.Folders.Find("SampleMasterCopyUs

masterCopyUserFolder.Name = "NewMasterCopyUserFolderName";
```




```
var masterCopyUserFolder =
ProjectLibrary.MasterCopyFolder.Folders.Find("SampleMasterCopyUserFolde

masterCopyUserFolder.SetAttributes(new[] {new KeyValuePair<string,object
("Name", "NewMasterCopyUserFolderName")});
```

See also

Accessing master copies

3.12 Accessing types

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project
- You have access to the required library.
  See Accessing global libraries.
- You have access to a group for types.
  See Accessing folders in a library.

Application

You can access the types contained in a library via the TIA Portal Openness API interface.

- You can enumerate the types.
- You can rename types.
- You can access the following attributes for every type:

Attribute	Data type	Description
Author	String	Returns the name of the author.
Comment	MultilingualText	Returns the comment.

Guid	Guid	Returns the GUID of the type.[1]
Name	String	Returns the name of the type. [2]

[1] You can find an individual type in a library using this attribute. The search is recursive.

[2] You can find an individual type in a folder using this attribute. Subfolders are not included in the search. A type name is not unique. There can be several types with the same name in different groups. However, the type Guid is unique.

Subclasses for library type objects

With TIA Portal Openness API you can access library type objects via sub-classes. The following subclasses are existing:

- Siemens.Engineering.Hmi.Faceplate.FaceplateLibraryType
- Siemens.Engineering.Hmi.Faceplate.FaceplateLibraryTypeVersion
- Siemens.Engineering.Hmi.RuntimeScripting.VBScriptLibraryType
- Siemens.Engineering.Hmi.RuntimeScripting.VBScriptLibraryTypeVersion
- Siemens.Engineering.Hmi.RuntimeScripting.CScriptLibraryType
- Siemens.Engineering.Hmi.RuntimeScripting.CScriptLibraryTypeVersion
- Siemens.Engineering.Hmi.Screen.ScreenLibraryType
- Siemens.Engineering.Hmi.Screen.ScreenLibraryTypeVersion
- Siemens.Engineering.Hmi.Screen.StyleLibraryType
- Siemens.Engineering.Hmi.Screen.StyleLibraryTypeVersion
- Siemens.Engineering.Hmi.Screen.StyleSheetLibraryTypeVersion
- Siemens.Engineering.Hmi.Tag.HmiUdtLibraryType
- Siemens.Engineering.Hmi.Tag.HmiUdtLibraryTypeVersion
- Siemens.Engineering.SW.Blocks.CodeBlockLibraryType
- Siemens.Engineering.SW.Blocks.CodeBlockLibraryTypeVersion
- Siemens.Engineering.SW.Types.PlcTypeLibraryType
- Siemens.Engineering.SW.Types.PlcTypeLibraryTypeVersion

The following code is an example of how to use the library type sub-classes

```
Copies the following program code to clipboard.
ProjectLibrary library = project.ProjectLibrary;
VBScriptLibraryType vbScriptType = ...;
```

```
VBScriptLibraryType libraryTypeAsVbScript = libraryType as
VBScriptLibraryType;

VBScriptLibraryTypeVersion libraryTypeVersionAsVbScript =
libraryTypeVersion as VBScriptLibraryTypeVersion
```

Program code

Modify the following program code to enumerate all types in the system folder of a library:

```
 Copies the following program code to clipboard.

public static void
EnumerateTypesInTypesSystemFolder(LibraryTypeSystemFolder
libraryTypeSystemFolder)

{

    foreach (LibraryType libraryType in
libraryTypeSystemFolder.Types)

    {

        //...

    }

}
```

Modify the following program code to enumerate all types in a user-defined folder of a library:

```
 Copies the following program code to clipboard.

public static void EnumerateTypesInTypesUserFolder
(LibraryTypeUserFolder libraryTypeUserGroup)

{

    foreach (LibraryType libraryType in
libraryTypeUserGroup.Types)

    {

        //...

    }

}
```

Modify the following program code to access the attributes of a type:

```
 Copies the following program code to clipboard.
```

```
public static void InspectPropertiesOfType (LibraryType
libTypeObject)
{
    string typeAuthor = libTypeObject.Author;
    MultilingualText typeComment = libTypeObject.Comment;
    string typeName = libTypeObject.Name;
    Guid typeGUID = libTypeObject.Guid;
}
```

Modify the following program code to find an individual type by its name or GUID:

```
 Copies the following program code to clipboard.
public static void FindTypeObjectInLibrary(ILibrary library)
{
    // Find type object by its GUID in a given library:
    System.Guid targetGuid = ...;
    LibraryType libTypeByGUID = library.FindType(targetGuid);
    // Find type object by its name in a given group:
    LibraryTypeFolder libTypeSystemFolder =
library.TypeFolder;
    LibraryType libTypeByName =
libTypeSystemFolder.Types.Find("myTypeObject");
}
```

Modify the following program code to rename a type:

```
 Copies the following program code to clipboard.
```

```
// Setting the name attribute
var type =
project.ProjectLibrary.TypeFolder.Types.Find("SampleTypeName");
type.Name = "NewTypeName";
//Setting the name attribute dynamically
var type =
project.ProjectLibrary.TypeFolder.Types.Find("SampleTypeName");
type.SetAttributes(new[] {new KeyValuePair<string,object>
("Name", "NewTypeName")});
```

3.13 Accessing type versions

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project
- You have access to the required library.
  See Accessing global libraries.
- You have access to a group for types.
  See Accessing folders in a library.

Introduction

You can use the TIA Portal Openness to perform the following functionalities:

- Access the type versions
- Enumerate the type versions of a type
- Determine the type to which a type version belongs
- Enumerate the instances of a type version.
- Create a new instance of a type version.
- Navigate from an instance to its connected version object.

As of TIA Portal Openness v20, the Siemens.Engineering.Library.Types namespace is added to access the attributes of a library type version.

The LibraryTypeVersion serves as a base class for the following classes in WinCC and STEP7:

Wincc	STEP7

• FaceplateLibraryTypeVersion • CScriptLibraryTypeVersion • VBScriptLibraryTypeVersion • ScreenLibraryTypeVersion • StyleLibraryTypeVersion • StyleSheetLibraryTypeVersion • HmiUdtLibraryTypeVersion	• CodeBlockLibraryTypeVersion • PlcTypeLibraryTypeVersion

You can access the following attribute for every type version:

Attribute	Data type	Description	Access
Author	String	Returns the name of the author.	Read
Comment	MultilingualText	Returns the comment.	Read
Guid	Guid	Returns the GUID of the type version.[1]	Read
ModifiedDate	DateTime	Returns the date and time at which the type version was set to the "`Committed`" status.	Read
State	LibraryTypeVersionState	Returns the status of the version: • `InWork`: Corresponds to the status "In progress" or "In testing" depending on the associated type. • `Committed`: Corresponds to the status "Released".	Read
TypeObject	LibraryType	Returns the type to which this type version belongs.	Read
VersionNumber	Version	Returns the version number as a three digit version identification, for example, "1.0.0".[2]	Read

Dependencies	Siemens.Engineering.Library.Types.LibraryTypeVersionAssociation
Dependents	Siemens.Engineering.Library.Types.LibraryTypeVersionAssociation
MasterCopiesContainingInstances	Siemens.Engineering.Library.MasterCopies.MasterCopyAssociation
IsDefault	bool

OriginalLibrary[3]	string	The name of the project or library from which the current version of the type was generated	Read

[1] You can find an individual type version in a library using this attribute.

[2] You can find an individual type version in a "LibraryTypeVersion" composition using this attribute.

[3] The OriginalLibrary is added in TIA Portal Openness v20

Program code: Accessing attribute of type versions

To access attributes of a library type versions modify the following program code:

```
Copies the following program code to clipboard.
private static void
GetLibraryTypeVersionProperties(LibraryTypeVersion
typeVersion)
{
    string author = typeVersion.Author;
    MultilingualText comment = typeVersion.Comment;
    Guid guid = typeVersion.Guid;
    DateTime modifiedDate = typeVersion.ModifiedDate;
    Siemens.Engineering.Library.Types.LibraryTypeVersionState
state = typeVersion.State;
    Siemens.Engineering.Library.Types.LibraryType typeObject
= typeVersion.TypeObject;
    Version versionNumber = typeVersion.VersionNumber;
    LibraryTypeVersionAssociation dependencies =
typeVersion.Dependencies;
    LibraryTypeVersionAssociation dependents =
typeVersion.Dependents;
    MasterCopyAssociation masterCopiesContainingInstances =
typeVersion.MasterCopiesContainingInstances;
    bool isDefault = typeVersion.IsDefault;
    string originalLibrary = typeVersion.OriginalLibrary;
}
```

Program code: Enumerate all type versions of a type

Modify the following program code:

```
Copies the following program code to clipboard.
//Enumerate the type versions of a type
public static void EnumerateVersionsInType(LibraryType
libraryType)
{
    foreach (LibraryTypeVersion libraryTypeVersion in
libraryType.Versions)
    {
        //...
    }
}
```

Program code: Accessing the attributes of a type version

Modify the following program code:

```
Copies the following program code to clipboard.
//Acessing the attributes of a type version
public static void
InspectPropertiesOfVersion(LibraryTypeVersion libTypeVersion)
{
    string versionAuthor = libTypeVersion.Author;
    MultilingualText versionComment = libTypeVersion.Comment;
    Guid versionGUID = libTypeVersion.Guid; DateTime
versionModifiedDate = libTypeVersion.ModifiedDate;
    LibraryTypeVersionState versionStateLibrary =
libTypeVersion.State;
    LibraryType versionParentObject =
libTypeVersion.TypeObject;
    Version versionNumber = libTypeVersion.VersionNumber;
}
```

Program code: Creating an instance of a type version

You can create a new instance of a type version. The following objects are supported:

- Blocks (FB/FC)
- PLC user data types
- Screens
- VB scripts
- NVT

An instance can be created of a type version from global library and project library. When you create the instance of a type version from a global library, the type version is first synchronized with the project library.

With TIA Portal Openness v20, the CreateFrom() method adds a new parameter updatePathsMode for the PlcBlock and PLCTypes to manage conflicts during library type instantiation.

The UpdatePathsMode (Siemens.Engineering.Library.Types.UpdatePathsMode) accepts the following enum value definitions:

Enum value	Description
UpdatePathsMode.UpdatePathsInTarget	If a path conflict occurs, adjust the target path to match the source path
If a path conflict occurs, adjust the target path to match the source path	If a path conflict occurs, keep the target path the same
UpdatePathsMode.ThrowIfPathsConflict	If a path conflict occurs, cancel the operation

A recoverable Exception will be thrown if an instance cannot be created in the target, then the possible reasons are:

- The library type version is in-work
- An instance of the library type version already exists in the target device

Modify the following program code:

```
Copies the following program code to clipboard.

VBScriptLibraryTypeVersion scriptVersion = ...;
VBScriptComposition vbscripts = ...;
//Using the CreateFrom method to create an instance of the
version in the VBScripts composition
```

```
VBScript newScript = vbscripts.CreateFrom(scriptVersion);
```

Modify the following program code:



```
ScreenLibraryTypeVersion screenVersion = ...;
ScreenComposition screens = ...;
//Using the CreateFrom method to create an instance of the
version in the screens composition
Screen newScreen = screens.CreateFrom(screenVersion);
```

Modify the following program code:



```
CodeBlockLibraryTypeVersion blockVersion = ...;
PlcBlockComposition blocks = ...;
//Using the CreateFrom method to create an instance of
//the version in the blocks composition
PlcBlock newBlock =
blocks.CreateFrom(blockVersion,Library.Types.UpdatePathsMode.UpdatePaths
```

Modify the following program code:



```
PlcTypeLibraryTypeVersion plcTypVersione=...;
PlcTypeComposition types=...;
//Using the CreateFrom method to create an instance of the version in th
composition
PlcType newType =
types.CreateFrom(plcTypeVersion,Library.Types.UpdatePathsMode.UpdatePath
```

Modify the following program code to create NVT file:



```
var nvtFromLibraryType =
MyProject.ProjectLibrary.TypeFolder.Types.Find("NVT_TYPE_LIBRARY_1.nvt")
var nvt_TYPE_LIBRARY_1V1 = nvtFromLibraryType.Versions[0];
var resultPlcDocument =
unit.TypeGroup.Documents.CreateFrom(nvt_TYPE_LIBRARY_1V1 as
```

```
PlcDocumentLibraryTypeVersion);
```

Program code: Determining uses of a type version

The following uses are distinguished for type versions:

- The type version uses other type versions from the library.

  Example: A user data type is used in a program block. The program block must have access to the user data type. This means the program block depends on the user data type.

  When you access the Dependencies attribute of CodeBlockLibraryVersion through `GetDependencies()` method, a list of LibraryTypeVersions are returned.

- The type is being used by another type version in the library.

  Example: A user data type is used in a program block. The program block must have access to the user data type. The user data type has the associated program block. The program block depends on the user data type.

  When you access the Dependents attribute of PlcTypeLibraryTypeVersion through `GetDependents()` method, a list of LibraryTypeVersions are returned.

Both attributes return a list that contains objects of the `LibraryTypeVersion` type. If there are no uses, an empty list is returned.

Note

If you use these attributes on type versions with the "`InWork`" status, an exception can be thrown.

Modify the following program code:

```
Copies the following program code to clipboard.
//Determine the uses of a type version in a library
public static void
GetDependenciesAndDependentsOfAVersion(LibraryTypeVersion
libTypeVersion)
{
    IList<LibraryTypeVersion> versionDependents =
libTypeVersion.Dependents();
    IList<LibraryTypeVersion> versionDependencies =
libTypeVersion.Dependencies();
}
```

Program code

Modify the following program code to determine the type to which a type version belongs:

```
public static void GetParentTypeOfVersion(LibraryTypeVersion
libTypeVersion)
{
    LibraryType parentType = libTypeVersion.TypeObject;
}
```

Modify the following program code to determine the master copies that contain instances of a type version:

```
public static void
GetMasterCopiesContainingInstances(LibraryTypeVersion
libTypeVersion)
{
    MasterCopyAssociation masterCopies =
libTypeVersion.MasterCopiesContainingInstances;
}
```

Modify the following program code to find an individual type version by its version number:

```
public static void FindVersionInLibrary(ILibrary library,
Guid versionGUID)
{
    LibraryTypeVersion libTypeVersionByVersionNumber =
library.FindVersion(versionGUID);
}
```

3.14 Accessing blocks located in library

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- Opening a project
  See Opening a project

Introduction

You can use the TIA Portal Openness to get version information from a library object without instantiating and compiling the object.

The following new Export Action will be provided on the LibraryTypeVersion engineering object to export the version content.

The action will create the Exported xml file at given exportFileInfo.



Exception will occur:

- In case of the user exceptions stating "The version data cannot be exported as it in in-work state" and the version to be exported is in "in test" state
- In case of the General user DataExchange for example "FileAlreadyExists" exception stating "The export cannot be made because the file 'D:*.xml' already exists".

Library Type Version enginnering Object

The following additions are made to the Library TypeVersion engineering Object:

1. Export action is provided on LibraryTypeVersion
2. Navigator to navigate to the version content browsable:
   - Navigator name : ContentObject
   - ReadPublicationLevel : System
   - Relation name: Engineering.Library.DefaultVersionContentObject
   - Base navigator will provide a default implementation to provide empty browsable collection as version content. Client can override this navigator and provide versioncontent.
3. "LibraryTypeName" and "LibraryTypeGuid" on LibraryTypeVersion object

Attributes	ReadPublicationLevel
LibraryTypeName	System
LibraryTypeGuid	System

Note

This is necessary because without this info you would not be able to understand that exported version info (in the exported xml file) belongs to which type.

Exported Content

Following content are available in the exported file:

- Library Version Info exported

Attributes	SimaticMLAAccess
Author	ReadWrite
Guid	ReadOnly
Modified Data	ReadOnly
VersionNumber	ReadWrite
LibraryTypeName	ReadOnly
Library TypeGuid	ReadOnly

Navigators Exported:

- Comment

Note

Attributes with SimaticMLAccess readonly get exported only if you choose the exportoption as "ExportOptions.WithReadOnly".

Program code

Modify the following program code to export version content information:



```
Copies the following program code to clipboard.

Guid g = new Guid("35ad9996-d6d7-40c9-989f-0f0c7e21a7b2");
//Block1
var version = m_Project.ProjectLibrary.FindType(g).Versions[0];
version.Export(new
FileInfo(@"D:\ExportCodeBlock.xml"),ExportOptions.WithReadOnly);
```

See also

Opening a project

3.15 Accessing instances

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project
- You have access to the required library.
  See Accessing global libraries.
- You have access to a group for types.
  See Accessing folders in a library.

Application

You can access instances of type versions via the TIA Portal Openness API interface.

Use the `FindInstances(IInstanceSearchScope searchScope)` method to find all instances of a type version.

You can use the `searchScope` parameter to specify the area of the project to be searched. The following classes implement the `IInstanceSearchScope` interface and can be used to search for instances:

- `PlcSoftware`
- `HmiTarget`

The method returns a list that contains objects of the `LibraryTypeInstanceInfo` type. If there are no instances, an empty list is returned.

Note

Instances of faceplates and HMI user data types are always linked to the associated type version.

Instances of all other objects, such as program blocks or screens, can be linked to a type version.

Enumerate the instances of a type version

Modify the following program code:

```
Copies the following program code to clipboard.
//Enumerate  the instances of a type version in the project
LibraryTypeVersion version = ...;
```

```
PlcSoftware plcSoftware = ...;

IInstanceSearchScope searchScope = plcSoftware as
IInstanceSearchScope;

if(searchScope==null)

{

    //No search possible

}

IList<LibraryTypeInstanceInfo> instanceInfos =
version.FindInstances(searchScope);

IEnumerable<IEngineeringObject> instances =
instanceInfos.Select(instanceInfo =>
instanceInfo.LibraryTypeInstance);
```

Navigate from an instance to its connected version object

Use the LibraryTypeVersion attribute of LibraryTypeInstanceInfo service to navigate from an instance to its connected version object.

The following objects provide the LibraryTypeInstanceInfo service:

- Blocks FB
- Blocks FC
- PLC user data types
- Screens
- VB scripts

If an instance object is not connected to a version object, then it will not provide the "LibraryTypeInstanceInfo" service.

 Copies the following program code to clipboard.

```
FC fc = ...;

//Using LibraryTypeInstanceInfo service

LibraryTypeInstanceInfo instanceInfo =
fc.GetService<LibraryTypeInstanceInfo>();

if(instanceInfo != null)

{

    LibraryTypeVersion connectedVersion =
instanceInfo.LibraryTypeVersion;
```

```
    FC parentFc = instanceInfo.LibraryTypeInstance as FC;
//parentFc == fc
}
```

3.16 Accessing master copies

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- You have access to the required library.

  See Accessing global libraries
- You have access to a group for master copies.

  See Accessing folders in a library

Introduction

The TIA Portal Openness API interface supports access to master copies in a global library and the project library:

- Creating master copies
- Enumerating master copies in system folders and user-defined folders
- Renaming master copies
- Querying information from master copies
- Querying information from objects in a master copy

Attribute	Data type	Description
Author	String	Returns the name of the author.
ContentDescriptions	MasterCopyContentDescriptionComposition	Returns a description for the content of the MasterCopy.
CreationDate	DateTime	Returns the creation date.

Name	String	Returns the name of the master copy.

Program code

Modify the following program code to enumerate all master copies in the system folder of a library:

```
Copies the following program code to clipboard.
public static void EnumerateMasterCopiesInSystemFolder
(MasterCopySystemFolder masterCopySystemFolder)
{
    foreach (MasterCopy masterCopy in
masterCopySystemFolder.MasterCopies)
    {
        //...
    }
}
```

Modify the following program code to access an individual mastercopy by using the find method:

```
Copies the following program code to clipboard.
...
    MasterCopySystemFolder systemFolder =
projectLibrary.MasterCopyFolder;
    MasterCopyComposition mastercopies =
systemFolder.MasterCopies;
    MasterCopy masterCopy = mastercopies.Find("Copy of ...");
...
```

Modify the following program code to enumerate master copy groups and subgroups:

```
Copies the following program code to clipboard.
private static void EnumerateFolder(MasterCopyFolder folder)
{
    EnumerateMasterCopies(folder.MasterCopies);
    foreach (MasterCopyUserFolder subFolder in
folder.Folders)
```

```
    {
        EnumerateFolder(subFolder); // recursion
    }
}
private static void
EnumerateMasterCopies(MasterCopyComposition masterCopies)
{
    foreach (MasterCopy masterCopy in masterCopies)
    {
        ...
    }
}
```

Modify the following program code to access a MasterCopyUserFolder by using the find method:

```
Copies the following program code to clipboard.
...
    MasterCopyUserFolderComposition userFolderComposition =
...
    MasterCopyUserFolder userFolder =
userFolderComposition.Find("Name of user folder");
...
```

Modify the following program code to rename a master copy:

```
Copies the following program code to clipboard.
//Setting the name attribute
var masterCopy =
projectLibrary.MasterCopyFolder.MasterCopies.Find("SampleMasterCopyName"
masterCopy.Name = "NewMasterCopyName";
//Setting the name attribute dynamically
var masterCopy =
projectLibrary.MasterCopyFolder.MasterCopies.Find("SampleMasterCopyName"
masterCopy.SetAttributes(new[] {new KeyValuePair<string,object>("Name",
"NewMasterCopyName")});
```

Querying information from master copies

Modify the following program code to get information of a master copy:

```
public static void GetMasterCopyInformation(MasterCopy
masterCopy)
{
    string author = masterCopy.Author;
    DateTime creationDate = masterCopy.CreationDate;
    string name = masterCopy.Name;
}
```

Querying information from objects in a master copy

The MasterCopy object contains a navigator called ContentDescriptions, which is a composition of MasterCopyContentDescriptions.

A master copy may contain multiple objects. The MasterCopy object contains ContentDescriptions for each object directly contained in the MasterCopy. If the master copy contains a folder which also contains some items, the MasterCopy object only contains one ContentDescription of the folder.

```
MasterCopy multiObjectMasterCopy = ...;
//Using ContentDescriptions
MasterCopyContentDescriptionComposition
masterCopyContentDescriptions =
multiObjectMasterCopy.ContentDescriptions;
MasterCopyContentDescription contentDescription=
masterCopyContentDescriptions.First();
string name = contentDescription.ContentName;
Type type = contentDescription.ContentType;
```

3.17 Create master copy from a project in library

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

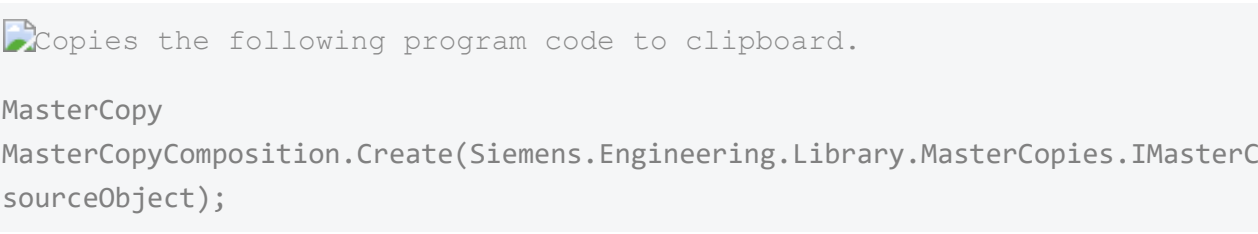
See Connecting to the TIA Portal

- A project is open.

  See Opening a project

Application

If the library is a read-write library, you can create a MasterCopy of an IMasterCopySource at target location.



An EngineeringException will be thrown if:

- The target location is read-only
- Creation of the MasterCopy from the source is rejected by the system

The following items are defined as IMasterCopySources:

- Device - HW
- DeviceItem - HW
- DeviceUserGroup - HW
- CodeBlock - SW
- DataBlock - SW
- PlcBlockUserGroup - SW
- PlcTag - SW
- PlcTagTable - SW
- PlcTagTableUserGroup - SW
- PlcType - SW
- PlcTypeUserGroup - SW
- PlcUnit - SW
- VBScript - HMI
- VBScriptUserFolder - HMI
- Screen - HMI
- ScreenTemplate - HMI
- ScreenTemplateUserFolder - HMI
- ScreenUserFolder - HMI

- Tag - HMI
- TagTable - HMI
- TagUserFolder - HMI

Program code

Use the following program code:

```
Copies the following program code to clipboard.

// create a master copy from a code block in the project library
public static void Create(Project project, PlcSoftware plcSoftware)
{
    MasterCopySystemFolder masterCopyFolder = project.ProjectLibrary.MasterCopyFolder;
    CodeBlock block = plcSoftware.BlockGroup.Groups[0].Blocks.Find("Block_1") as CodeBlock;
    MasterCopy masterCopy = masterCopyFolder.MasterCopies.Create(block);
}
```

3.18 Cleaning up library

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Introduction

You can use the TIA Portal Openness application to remove unused version of the specified types in library, or all the types within a specified folder in a library.

The CleanUpLibrary operation is available for both project and user global libraries for which libraries must be read-write mode.

Method

The CleanUpLibrary method is an overloaded function that supports different argument values for both project-specific and global libraries.

Library type	CleanUpLibrary Signature
Project library	void CleanUpLibrary(IEnumerable<ILibraryTypeOrFolderSelection> selectedTypesAndFolders, CleanupLibraryMode cleanupLibraryMode)
User global library	void CleanUpLibrary(IEnumerable<ILibraryTypeOrFolderSelection> selectedTypesAndFolders)

In project library, the CleamUpLibrary() cleanup the version based on two CleanupLibraryMode flags:

CleanupLibraryMode- flags	Description
CleanupLibraryMode.DeleteUnusedTypes	Deletes the type if the none of its versions are used in the project.
CleanupLibraryMode.PreserveDefaultVersionOfUnusedTypes	The highest version or default version of type will be exempted from deletion. Types will not be deleted completely

In global library, the CleanUpLibrary() by default performs cleanup of Global Library without deleting the complete type.

Criteria for version cleanup

A version is eligible for cleanup if it meets the following criteria:

Mode	Criteria
General	The version is not in use (exception to this rule if version is being used by a version that is also in the selection to be cleaned up AND is eligible for cleanup)
	The version is not in-test

General	The version is not the base version for an in-test version
	There doesn't exist an instance of the version in the project or master copy folder
PreserveDefaultVersionOfUnusedTypes mode	The version meets the general criteria
	The version is not highest version or default version of type
DeleteUnusedTypes mode	The version meets the general criteria

Program code

To use cleanup action for a subset of types and folders in Project Library modify the following program code:



```
Copies the following program code to clipboard.
private static void CleanUpLibraryForProjectLibrary()
{
    TiaPortal tiaPortal = LaunchOrAttachTiaPortal();
    var projectLibrary =
tiaPortal.Projects[0].ProjectLibrary;
    CleanUpMode cleanUpMode = CleanUpMode.DeleteUnusedTypes;
    var libType =
projectLibrary.TypeFolder.Types.Find("TypeName");
    LibraryTypeUserFolder userFolderContainingTypes =
projectLibrary.
    TypeFolder.Folders.Find("Userfolder");
    projectLibrary.CleanUpLibrary(new
ILibraryTypeOrFolderSelection[]
    { libType, userFolderContainingTypes }, cleanUpMode);
}
```

To invoke cleanup action for all types in a Project library modify the following program code:



```
Copies the following program code to clipboard.
private static void CleanUpLibraryForAllTypes()
```

```
{
    TiaPortal tiaPortal = LaunchOrAttachTiaPortal();
    var projectLibrary =
tiaPortal.Projects[0].ProjectLibrary;
    CleanUpMode cleanUpMode = CleanUpMode.DeleteUnusedTypes;
    var libType =
projectLibrary.TypeFolder.Types.Find("TypeName");
    LibraryTypeUserFolder userFolderContainingTypes =
projectLibrary.
    TypeFolder.Folders.Find("Userfolder");
    projectLibrary.CleanUpLibrary(new[] {
projectLibrary.TypeFolder }, cleanUpMode);
}
```

To invoke cleanup action for a subset of types and folders in Global Library modify the following program code:

```
 Copies the following program code to clipboard.
private static void CleanUpForGlobalLibrary()
{
    TiaPortal tiaPortal = LaunchOrAttachTiaPortal();
    UserGlobalLibrary userGlobalLib =
tiaPortal.GlobalLibraries.
    Open(new FileInfo(@"SomePath"), OpenMode.ReadWrite);
    var libType =
userGlobalLib.TypeFolder.Types.Find("TypeName");
    LibraryTypeFolder userFolderContainingTypes =
    userGlobalLib.TypeFolder.Folders.Find("Userfolder");
    userGlobalLib.CleanUpLibrary(new
ILibraryTypeOrFolderSelection[]{ libType,
    userFolderContainingTypes })
}
```

To invoke cleanup action for all types in a global library modify the following program code:

```
 Copies the following program code to clipboard.
```

```
private static void CleanUpForGlobalLibrary()
{
    TiaPortal tiaPortal = LaunchOrAttachTiaPortal();
    UserGlobalLibrary userGlobalLib =
tiaPortal.GlobalLibraries.
    Open(new FileInfo(@"SomePath"), OpenMode.ReadWrite);
    var libType =
userGlobalLib.TypeFolder.Types.Find("TypeName");
    LibraryTypeFolder userFolderContainingTypes =
    userGlobalLib.TypeFolder.Folders.Find("Userfolder");
    userGlobalLib.CleanUpLibrary(new[] {
userGlobalLib.TypeFolder });
}
```

3.19 Create an object from a master copy

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- PLC is not online.

Introduction

The TIA Portal Openness API interface supports the use of master copies in the project. You can create an object in the object's composition from a master copy in a project library or a global library using the CreateFrom method.

The return type will correspond to the respective composition's return type.

The CreateFrom method only supports master copies containing single objects. If the composition where the action is called and the source master copy are incompatible (e.g. source master copy contains a plc tag table and the composition is a plc block composition), a recoverable exception will be thrown.

The following compositions are supported:

- Siemens.Engineering.HW.DeviceComposition
- Siemens.Engineering.HW.DeviceItemComposition
- Siemens.Engineering.SW.Blocks.PlcBlockComposition

- Siemens.Engineering.SW.Tags.PlcTagTableComposition
- Siemens.Engineering.SW.Tags.PlcTagComposition
- Siemens.Engineering.SW.Types.PlcTypeComposition
- Siemens.Engineering.SW.PlcDocumentComposition
- Siemens.Engineering.SW.TechnologicalObjects.TechnologicalInstanceDBComposition
- Siemens.Engineering.SW.Tags.PlcUserConstantComposition
- Siemens.Engineering.Hmi.Tag.TagTableComposition
- Siemens.Engineering.Hmi.Tag.TagComposition
- Siemens.Engineering.Hmi.Screen.ScreenComposition
- Siemens.Engineering.Hmi.Screen.ScreenTemplateComposition
- Siemens.Engineering.Hmi.RuntimeScripting.VBScriptComposition
- Siemens.Engineering.HW.SubnetComposition
- Siemens.Engineering.HW.DeviceUserGroupComposition
- Siemens.Engineering.SW.Blocks.PlcBlockUserGroupComposition
- Siemens.Engineering.SW.ExternalSources.PlcExternalSourceUserGroupComposition
- Siemens.Engineering.SW.Tags.PlcTagTableUserGroupComposition
- Siemens.Engineering.SW.Types.PlcTypeUserGroupComposition

Program code: Create a PLC block from a mastercopy

Modify the following program code to create an PLC block from a master copy in a library:

```
Copies the following program code to clipboard.

var plcSoftware = ...;
MasterCopy copyOfPlcBlock = ...;
PlcBlock
plcSoftware.BlockGroup.Blocks.CreateFrom(copyOfPlcBlock);
```

Program code: Create a device from a mastercopy

Modify the following program code to create a device from a master copy in a library:

```
Copies the following program code to clipboard.

Project project = ...;
MasterCopy copyOfDevice = ...;
Device newDevice = project.Devices.CreateFrom(copyOfDevice);
```

Program code: Create a device item from a mastercopy

Modify the following program code to create a device item from a master copy in a library:

```
Copies the following program code to clipboard.

Device device = ...;

MasterCopy copyOfDeviceItem = ...;

DeviceItem newDeviceItem =
device.DeviceItems.CreateFrom(copyOfDeviceItem);
```

Program code: Create a subnet from a mastercopy

Modify the following program code to create a subnet from a master copy in a library:

```
Copies the following program code to clipboard.

Project project = ...;

MasterCopy copyOfSubnet = ...;

Subnet newSubnet = project.Subnets.CreateFrom(copyOfSubnet);
```

Program code: Create a device folder from a mastercopy

Modify the following program code to create a device folder from a master copy in a library:

```
Copies the following program code to clipboard.

Project project = ...;

MasterCopy copyOfDeviceGroup = ...;

DeviceGroup newDeviceGroup=
project.DeviceGroups.CreateFrom(copyOfDeviceGroup);
```

Program code: Create a NVT document from a mastercopy

Modify the following program code to create a NVT document from a master copy in a library

```
Copies the following program code to clipboard.

var nvtFromMasterCopies =
MyProject.ProjectLibrary.MasterCopyFolder.MasterCopies.Find("NVT_Master
var resultPlcDocument = unit.TypeGroup.Documents.CreateFrom(nvtFromMaste
```

See also

Accessing master copies

3.20 Copying master copies

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Introduction

The TIA Portal Openness API interface supports copying of master copies within a library and between libraries using the CreateFrom action.

The action will create a new object based on the source master copy and place it in the composition where the action was called. The action will try to create the new master copy with the same name as the source master copy.

If such name is not available, the system will give the new master copy a new name. It will then return the new master copy.

If the composition where the "CreateFrom" action is called is in a read-only global library, a recoverable exception will be thrown.

Program code

Modify the following program code:

```
Copies the following program code to clipboard.

Project project = ...;
ProjectLibrary projectLibrary = project.ProjectLibrary;
MasterCopy copiedMasterCopy =
projectLibrary.MasterCopyFolder.MasterCopies.CreateFrom(sampleMasterCopy
```

See also

Accessing master copies

3.21 Creating new type from document

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Introduction

You can use the TIA Portal Openness application to create specific library types into TIA Portal project using the source document.

The source provides you the directory information which includes the files (SimaticML,WinCCML, S7DCL, SCL, STL, UDT, NVT), filename, and target environment.

The API is available in Project Library and not supported via Global Library. An engineering exception is thrown if the API is used from Global Library.

Method, Parameter, and Enum

To create a successful library type creation from the document, you can use the CreateFromDocuments() method which accepts the following parameters:

Parameter name	Data type	Is Mandatory	Description
dirInfo	System.IO.DirectoryInfo	Yes	Document directory informatio
fileNameWithoutExtension	System.String	Yes	Document name with out extension.
targetEnvironment	Siemens.Engineering.Library.IEngineeringObject	No	Target environme can be a PlcBlockGr or a PlcTypeGro where the instance g created in project(on applicable Step7 type

libraryImportOption[1]	Siemens.Engineering.Library.LibraryImportOptions	Yes	Enum is used to import culture specific content, the purpose of this enum is to control the project languages, not the library comments. The WinCC does not have the instance concept, so it does not modify project languages when imported to Library. In Library, the TypeObject or VersionObject comments are not multilingual.

The libraryImportOption accepts the following enum definitions:

Enum value	Description
LibraryImportOptions.None	Default behavior from the TIA Portal Openness (Step7) is used, which will be different for different document formats. This behavior is defined by the client, not from Library.
LibraryImportOptions.SkipInactiveCultures	Skip the cultures and texts If the cultures indicated in file used for Import are not

	active in the TIA Portal project where it is being imported.
LibraryImportOptions.ActivateInactiveCultures	Activate the cultures and import the texts as well, if the cultures indicated in file used for Import are valid and supported but are not active in the TIA Portal project where it is being imported.

Note

The enum values SkipInActiveCultures and ActivateInactiveCultures are not supported by the TIA Portal Openness in V20. The EngineeringTargetInvocation exception is thrown if these are used to invoke the API.Once the functionality for SkipInActiveCultures and ActivateInactiveCultures added by the TIA Portal Openness, this check will be removed.

The CreateFromDocuments() returns Siemens.Engineering.Library.TypeCreateTransferResults that contains the list of details:

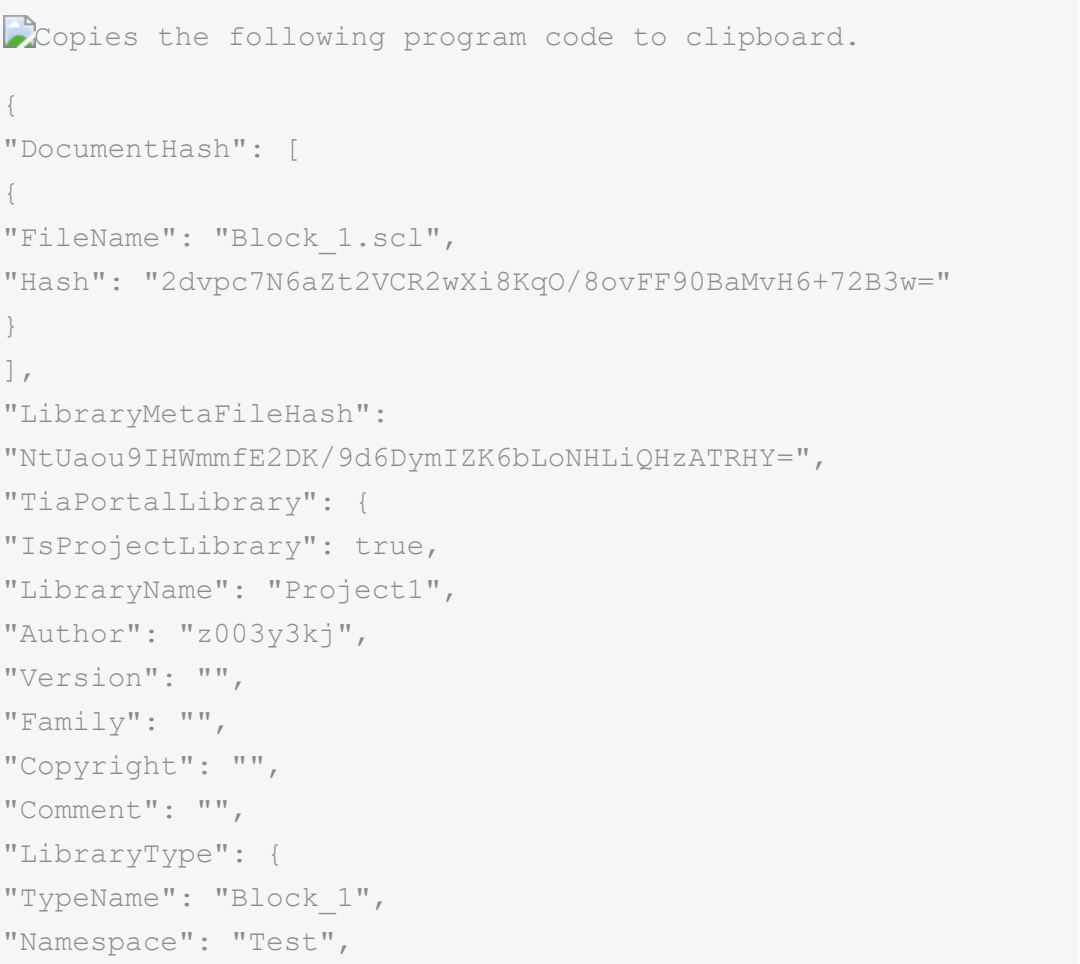
Data type	Description
TypeCreateTransferResults.Messages	Browse through collection of log messages generated during version import. The Messages is TransferResultMessageComposition that is composed of list of TransferResultMessage. The TransferResultMessage has log message details.
TypeCreateTransferResults.CreatedType	Returns the newly created Library type, which contains the default version in InWork mode.
TypeCreateTransferResults.TransferResultState	Detailed information on state of the create version from document. The TransferResultState is Enum has Success or Warning. An engineering exception is thrown in case of an error state, which requires a roll back. So the enum value doesn't represent Error state

TypeCreateTransferResults.Parent	Parent is IEngineeringObject

Exception

An EngineeringTargetInvocation is thrown in the following scenarios:

- if any of the mandatory parameter is not passed.
- if document is not handled by any TIA Portal Openness
- if Step7 related documents are passed into HMI API and vice versa.
- if a wrong IEngineeringObject (TargetEnvironment) is passed.
- if APIs are used interchangeably between Step7 and HMI.
- if directory have multiple files with supported extensions matching provided file name, then no new type and version gets created. For example, the provided filename is 'Block_1' directory contains 'Block_1.xml', 'Block_1.s7dcl'.
- created type must have following properties: Namespace, TypeGuid, and DoNotUse based on version exported library meta file.
- newly created type version must have "MinimumTargeDeviceVersion" based on version exported library meta file. (Applicable for WinCC Unified MTDV supported types only). For example, a sample of library meta file below:



```
"TypeGuid": "45e9f803-eb80-4171-af9e-46796145a784",
"DoNotUse": false,
"Folder": "Types"
},
"LibraryVersion": {
"VersionGuid": "691ee24b-517b-44a7-863b-5bac1ec91163",
"VersionNumber": "0.0.1",
"Author": "z003y3kj",
"IsDefault": true,
"InWork": false,
"VersionComment": "",
"DependsOn": [],
"MinimumTargetDeviceVersion": ""
}
}
}
```

Program code

To create a library type from document for HMI modify the following program code



```
Copies the following program code to clipboard.

private void CreateLibraryTypeFromDocumentForHMI()
{
    using (TiaPortal portal = new
TiaPortal(TiaPortalMode.WithoutUserInterface))
    {
    Console.WriteLine("TIA portal connection is successful");

    Project project = portal.Projects.Open(new
System.IO.FileInfo
    (@"D:\Temp\Test\Project1_test2\Project1_test2.ap20"));
    DirectoryInfo importLocation = new
DirectoryInfo(@"D:\Temp\Test");
    const string fileName = "TestFile";
    Console.WriteLine("Calling Create from document on
version composition");
    try
```

```
    {
    TypeCreateTransferResults typeCreateTransferResults =
project.ProjectLibrary.
    TypeFolder.Types.CreateFromDocuments(importLocation,
fileName, LibraryImportOptions.Override);
    if(typeCreateTransferResults.TransferResultState ==
TransferResultState.Success)
    {
    LbraryType createdType =
typeCreateTransferResults.CreatedType;
    Console.WriteLine("Type created successfully");
    }
    else
    {

    Console.WriteLine("Type creation failed.");
    foreach (TransferResultMessageComposition
transferResultMessage in typeCreateTransferResults.Messages)
    {
    Console.WriteLine(transferResultMessage.Message);
    }
    }
    project.Save();
    project.Close();
    }
    catch (EngineeringException e)
    {
    Console.WriteLine(e);
    }
}
```

To create a specific library type from document for STEP7 modify the following program code:



```
private void CreateLibraryTypeFromDocumentForStep7()
{
using (TiaPortal portal = new
TiaPortal(TiaPortalMode.WithoutUserInterface))
{
    Console.WriteLine("TIA portal connection is successful");
    Project project = portal.Projects.Open
    (new
System.IO.FileInfo(@"D:\Temp\Test\Project1_test2\Project1_test2.ap20"));
    DirectoryInfo importLocation = new DirectoryInfo(@"D:\Temp\Test");
    const string fileName = "TestFile";
    // Get Device
    Device device = GetDevice()
    //Get DeviceItem
    DeviceItem rack = GetDeviceItem(Device device)
    //Get DeviceItem
    DeviceItem plc = GetDeviceItem(DeviceItem rack);
    SoftwareContainer softwareContainer = plc
.GetService<SoftwareContainer>();
    PlcSoftware plcSoftware = softwareContainer?.Software as
PlcSoftware;
    PlcBlockSystemGroup blockGroup = plcSoftware.BlockGroup;
    //For by copy type test environment is required.
    Console.WriteLine("Calling Create from document on version
composition");
    try
    {
    TypeCreateTransferResults typeCreateTransferResults =
project.ProjectLibrary.
    TypeFolder.Types.CreateFromDocuments(importLocation, fileName,
blockGroup, LibraryImportOptions.Override);
    if(typeCreateTransferResults.TransferResultState ==
TransferResultState.Success)
```

```
    {
    LbraryType createdType =
typeCreateTransferResults.CreatedType;
    Console.WriteLine("Type created successfully");
    }
    else
    {
    Console.WriteLine("Type creation failed.");
    foreach (TransferResultMessageComposition
transferResultMessage in typeCreateTransferResults.Messages)
    {
    Console.WriteLine(transferResultMessage.Message);
    }
    }
    Console.WriteLine("Type created successfully");
    project.Save();
    project.Close();
    }
    catch (EngineeringException e)
    {
    Console.WriteLine(e);
    }
}
```

3.22 Updating type from document

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Introduction

You can use the TIA Portal Openness application to create specific library types into TIA Portal project using the source document.

The source provides you the directory information which includes the files (SimaticML,WinCCML, S7DCL, SCL, STL, UDT), filename, and target environment.

The API is available in Project Library and not supported via Global Library. An engineering exception is thrown if the API is used from Global Library.

Method, Parameter, and Enum

To create a successful library type creation from the document, you can use the CreateFromDocuments() method which accepts the following parameters:

Parameter name	Data type	Is Mandatory	Description
dirInfo	System.IO.DirectoryInfo	Yes	Document directory informatio
fileNameWithoutExtension	System.String	Yes	Document name with out extension.
targetEnvironment	Siemens.Engineering.Library.IEngineeringObject	No	Target environme can be a PlcBlockGr or a PlcTypeGro where the instance g created in project(on applicable Step7 type

createOptions	Siemens.Engineering.Library.Types.CreateOptions	Yes	Options for you to decide during the version creation if existing in-work version needs to be overridden based on library meta file version number. Using CreateOptions.None will throw exception if already in-work version exist.
libraryImportOption[1]	Siemens.Engineering.Library.LibraryImportOptions	Yes	Enum is used to import culture specific content, the purpose of this enum is to control the project languages, not the library comments. The WinCC does not have the instance concept, so it does not modify project languages when imported to Library. In Library, the TypeObject or VersionObject comments are not multilingual.

For the definition of the libraryImportOption enum value and return types from the CreateFromDocuments() method, refer to Creating new type from document -> **Method, Parameter, and Enum** section.

Exception

An EngineeringTargetInvocation is thrown in the following scenarios:

- if any of the mandatory parameter is not passed.
- if document is not handled by any client.

- If directory have multiple files with supported extensions matching provided file name, then no new version gets created and exception is thrown. For example, the provided filename is 'Block_1'
  directory contains 'Block_1.xml', 'Block_1.s7dcl'.
- newly created version must have "MinimumTargeDeviceVersion" based on version exported library meta file. This is applicable only for WinCC Unified MTDV Supported Types. For example, a sample of library meta file below:



```
{
"DocumentHash": [
{
"FileName": "Block_1.scl",
"Hash": "2dvpc7N6aZt2VCR2wXi8KqO/8ovFF90BaMvH6+72B3w="
}
],
"LibraryMetaFileHash":
"NtUaou9IHWmmfE2DK/9d6DymIZK6bLoNHLiQHzATRHY=",
"TiaPortalLibrary": {
"IsProjectLibrary": true,
"LibraryName": "Project1",
"Author": "z003y3kj",
"Version": "",
"Family": "",
"Copyright": "",
"Comment": "",
"LibraryType": {
"TypeName": "Block_1",
"Namespace": "Test",
"TypeGuid": "45e9f803-eb80-4171-af9e-46796145a784",
"DoNotUse": false,
"Folder": "Types"
},
"LibraryVersion": {
"VersionGuid": "691ee24b-517b-44a7-863b-5bac1ec91163",
"VersionNumber": "0.0.1",
"Author": "z003y3kj",
"IsDefault": true,
"InWork": false,
"VersionComment": "",
"DependsOn": [],
```

```
"MinimumTargetDeviceVersion": ""
}
}
}
```

Program code

To create a library type from document for HMI modify the following program code:



```
Copies the following program code to clipboard.

private void CreateLibraryTypeFromDocumentForHMI()
{
    using (TiaPortal portal = new
TiaPortal(TiaPortalMode.WithoutUserInterface))
    {
    Console.WriteLine("TIA portal connection is successful");
    Project project = portal.Projects.Open(new
System.IO.FileInfo(@"D:\Temp\Test\Project1_test2\Project1_test2.ap20"));
    DirectoryInfo importLocation = new DirectoryInfo(@"D:\Temp\Test");
    const string fileName = "TestFile";
    try
    {
    // Get library type
    LibraryType libraryType = GetLibraryType();
    LibraryTypeVersionComposition versionComposition = type.Versions;
    VersionCreateTransferResults versionCreateTransferResult =
versionComposition.
    CreateFromDocuments(directoryInfo, fileName,
CreateOptions.Override,LibraryImportOptions.Override);
    if(versionCreateTransferResult.TransferResultState ==
TransferResultState.Success)
    {
    LbraryTypeVersion version =
versionCreateTransferResult.CreatedVersion;
    Console.WriteLine("Version created successfully");
```

```
    }
    else
    {
    Console.WriteLine("Version creation failed.");
    foreach (TransferResultMessageComposition
transferResultMessage in
versionCreateTransferResult.Messages)
    {
    Console.WriteLine(transferResultMessage.Message);
    }
    }
    project.Save();
    project.Close();
    }
    catch (EngineeringException e)
    {
    Console.WriteLine(e);
    }
    }
    private LibraryType GetLibraryType()
    {
    return project.ProjectLibrary.TypeFolder.Types[1];
    }
}
```

To create a specific library type from document for STEP7 modify the following program code:

```
 Copies the following program code to clipboard.
private void CreateLibraryTypeFromDocumentForStep7()
{
using (TiaPortal portal = new
TiaPortal(TiaPortalMode.WithoutUserInterface))
{
```

```
    Console.WriteLine("TIA portal connection is successful");
    Project project = portal.Projects.Open
    (new
System.IO.FileInfo(@"D:\Temp\Test\Project1_test2\Project1_test2.ap20"));
    DirectoryInfo importLocation = new DirectoryInfo(@"D:\Temp\Test");
    const string fileName = "TestFile";
    // Get Device
    Device device = GetDevice()
    //Get DeviceItem
    DeviceItem rack = GetDeviceItem(Device device)
    //Get DeviceItem
    DeviceItem plc = GetDeviceItem(DeviceItem rack);
    SoftwareContainer softwareContainer = plc
.GetService<SoftwareContainer>();
    PlcSoftware plcSoftware = softwareContainer?.Software as
PlcSoftware;
    PlcBlockSystemGroup blockGroup = plcSoftware.BlockGroup;
    //For by copy type test environment is required.
    Console.WriteLine("Calling Create from document on version
composition");
    try
    {
    TypeCreateTransferResults typeCreateTransferResults =
project.ProjectLibrary.
    TypeFolder.Types.CreateFromDocuments(importLocation, fileName,
blockGroup,CreateOptions.Override,LibraryImportOptions.Override);
    if(typeCreateTransferResults.TransferResultState ==
TransferResultState.Success)
    {
    LbraryType createdType = typeCreateTransferResults.CreatedType;
    Console.WriteLine("Type created successfully");
    }
    else
```

```
    {
    Console.WriteLine("Type creation failed.");
    foreach (TransferResultMessageComposition
transferResultMessage in typeCreateTransferResults.Messages)
    {
    Console.WriteLine(transferResultMessage.Message);
    }
    }
    Console.WriteLine("Type created successfully");
    project.Save();
    project.Close();
    }
    catch (EngineeringException e)
    {
    Console.WriteLine(e);
    }
}
```

3.23 Determining out-of-date type instances

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- You have access to the required library.
  See Accessing global libraries
- You have access to a folder for types.
  See Accessing folders in a library.

Application

The TIA Portal Openness API interface allows you to determine type versions which belong to the instances in the open project. The TIA Portal Openness API returns one of the following two states per instance:

- The instance refers to an out-of-date type version.

- The instance refers to the latest type version.

The following rules apply when determining the version:

- You determine the version based on a library and the project that you want to open via the TIA Portal Openness API interface.
- Instances are not updated when you determine the version.

Signature

Use the `UpdateCheck` method to determine the instances of a type version:

```
UpdateCheck(Project project, UpdateCheckMode updateCheckMode)
```

Parameter	Function
Project	Specifies the project in which the type versions of instances are determined.
UpdateCheckMode	Specifies the versions that are determined: • `ReportOutOfDateOnly`: Returns only status of the "out of date" type. • `ReportOutOfDateAndUpToDate`: Returns status of the type "out of date" and "up to date".

Result

The devices of the project are scanned from top to bottom when determining the version. Each device is checked to determine whether its configuration data contain an instance of a type version from the specified library. The `UpdateCheck` method returns the result of the version check in hierarchical order.

The table below shows a result of a version check with the parameter `UpdateCheck.ReportOutOfDateAndUpToDate`:

Update check for: HMI_1

	Update check for library element Screen_1 0.0.3		
		Out-of-date	
			\HMI_1\Screens Screen_4 0.0.1 \HMI_1\Screens Screen_2 0.0.2
		Up-to-date	
			\HMI_1\Screens Screen_1 0.0.3 \HMI_1\Screens Screen_10 0.0.3
Update check for: HMI_2			
	Update check of library element Screen_4 0.0.3		
		Out-of-date	
			\Screens folder1 Screen_02 0.0.1 \Screens folder1 Screen_07 0.0.2
		Up-to-date	
			\Screens folder1 Screen_05 0.0.3 \Screens folder1 Screen_08 0.0.3

Program code

There is no difference between project and global libraries in handling the update check.

Modify the following program code to determine the type versions from a global or project library for instances in the project:

```
Copies the following program code to clipboard.

public static void UpdateCheckOfGlobalLibrary(Project
project, ILibrary library)

{

    // check for out of date instances and report only out of
date instances in the returned feedback

    UpdateCheckResult result = library.UpdateCheck(project,
UpdateCheckMode.ReportOutOfDateOnly);
```

```
    //Alternatively, check for out of date instances and
report both out of date and up to date instances in the
returned feedback
    UpdateCheckResult alternateResult =
library.UpdateCheck(project,
UpdateCheckMode.ReportOutOfDateAndUpToDate);
    //Show result
    RecursivelyWriteMessages(result.Messages);


    // Alternatively, show result and access single message
parts
    RecursivelyWriteMessageParts(result.Messages);
}
```

Modify the following program code to output the result of the version check and process the messages individually:

```
Copies the following program code to clipboard.
private static void RecursivelyWriteMessages
(UpdateCheckResultMessageComposition messages, string indent
= "")
{
    indent += "\t";
    foreach (UpdateCheckResultMessage message in messages)
    {
        Console.WriteLine(indent + message.Description);
        RecursivelyWriteMessages(message.Messages, indent);
    }
}
```

Modify the following program code to access individual message parts in the result of the version check:

```
Copies the following program code to clipboard.
private static void RecursivelyWriteMessageParts
(UpdateCheckResultMessageComposition messages, string indent=
"")
```

```
{
    indent += "\t";
    foreach (UpdateCheckResultMessage message in messages)
    {
        Console.WriteLine(indent + "Full description: " +
message.Description);
        foreach (KeyValuePair<string, string> messagePart in
message.MessageParts)
        {
             // first level
             // part 1: device name
             // second level:
             // part 1: Name of the type in the global
library
             // part 2: version of the type in the global
library
             // third level:
             // part 1: title (either "Out-of-date" or "Up-
to-date");
             // fourth level:
             // part 1: Path hierarchy to instance
             // part 2: Instance name in project
             // part 3: Version of the instance in the
project
             Console.WriteLine(indent + "*Key: {0} Value:
{1}", messagePart.Key, messagePart.Value);
        }
     RecursivelyWriteMessageParts(message.Messages,indent);
    }
}
```

3.24 Updating the project

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project
- You have access to the required library.
  See Accessing global libraries.
- You have access to a folder for types.
  See Accessing folders in a library.

Application

The TIA Portal Openness API interface allows you to update instances of a selected type within a type folder in a project.

When updating, the instances used in the project are updated based on the last released type version. If you start updating the instances from a global library, synchronization is performed beforehand.

Signature

Use the `UpdateProject` method to update instances.

Use the following call for classes which implement the `LibraryTypes` interface:

```
void UpdateProject(IUpdateProjectScope updateProjectScope)
```

Use the following call for classes which implement the `ILibrary` interface:

```
void UpdateProject(IEnumerable<ILibraryTypeOrFolderSelection>
selectedTypesOrFolders, IEnumerable <IUpdateProjectScope>
updateProjectScope)
```

Each call is entered in the log file in the project directory.

Parameter	Function
`IEnumerable<ILibraryTypeOrFolderSelection> selectedTypesOrFolders`	Specifies the folder or types to be synchronized or their instances in the project to be updated.
`IUpdateProjectScope updateProjectScope` `IEnumerable <IUpdateProjectScope> updateProjectScope`	Specifies the object(s) in the project in which the uses of instances are to be updated. The following objects are supported: • `PlcSoftware` • `HmiTarget`

Program code

Modify the following program code to update instances of selected types within a type folder:

```
Copies the following program code to clipboard.
private static void UpdateInstances(ILibrary myLibrary,
LibraryTypeFolder singleFolderContainingTypes, LibraryType
singleType, PlcSoftware plcSoftware, HmiTarget hmiTarget)
{
    //Update Instances of multiple types (subset of types and
folders)
    IUpdateProjectScope[] updateProjectScopes =
    {
        plcSoftware as IUpdateProjectScope, hmiTarget as
IUpdateProjectScope
    };
    myLibrary.UpdateProject(new
ILibraryTypeOrFolderSelection[] {singleType,
singleFolderContainingTypes}, updateProjectScopes);
    //Update Instances of multiple types (all types in
library)
    myLibrary.UpdateProject(new[] {myLibrary.TypeFolder},
updateProjectScopes);
}
```

3.25 Updating a library

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project
- You have access to the required library.
  See Accessing global libraries.
- You have access to a folder for types.
  See Accessing folders in a library.

Application

The TIA Portal Openness API interface supports the following updates in the project library:

- Synchronize selected types between libraries.

The folder structure is not adapted when you perform synchronization. The types to be updated are identified by their GUID and updated:

- If a type in a library includes a type version that is missing in the library to be updated, the type version is copied.
- If a type in a library includes a type version with the different GUID, the process is aborted and an Exception is thrown.

Signature

Use the `UpdateLibrary` method to synchronize type versions.

Use the following call for classes which implement the `LibraryTypes` interface:

```
void UpdateLibrary(ILibrary targetLibrary)
```

Use the following call for classes which implement the `ILibrary` interface:

```
void UpdateLibrary(IEnumerable<LibraryTypeOrFolderSelection>
selectedTypesOrFolders, ILibrary targetLibrary)
```

Parameter	Function
`IEnumerable<ILibraryTypeOrFolderSelection> selectedTypesOrFolders`	Specifies the folder or types to be synchronized or their instances in the project to be updated.
`ILibrary targetLibrary`	Specifies the library whose contents will be synchronized with a library. If source library and destination library are identical, an exception is thrown.

Program code

Modify the following program code to synchronize a type from the project library with a global library:

```
Copies the following program code to clipboard.

sourceType.UpdateLibrary(projectLibrary);
```

Modify the following program code to synchronize selected types within a type folder between a global library and the project library:

```
Copies the following program code to clipboard.

globalLibrary.UpdateLibrary(new[]{globalLibrary.TypeFolder},
projectLibrary);
```

3.26 Cleaning up library

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness application to delete the unused versions of the specified types in a library and delete all the types in a specified folder in a library.

The Cleanup() is available for project and global libraries for which libraries must be read/write. This feature is analogus to the Clean up Library feature present in TIA Portal user interface.

The Cleanup() performs function based on the specified CleanupLibraryMode flag.

A version is cleaned up based on the specified behavior of the two CleanupLibraryModes:

CleanupLibraryMode	Description
CleanupLibraryMode.AllowTypeDeletion	Allow deletion of all the versions on a type to be cleaned up. If all versions on a type have been cleaned up, the type too will be cleaned up.
CleanupLibraryMode.PreserveHighestVersion	Preserve the highest version on a type

Program code

Modify the following program code to invoke cleanup action for a subset of types and folders:

```
Copies the following program code to clipboard.

...
ILibrary myLibrary = ...;
```

```
CleanupLibraryMode cleanupLibraryMode = ...;
ILibraryType typeA = ...;//from myLibrary
ILibraryType typeB = ...; //from myLibrary
LibraryTypeFolder singleFolderContainingTypes = ...;//from
myLibrary
myLibrary.Cleanup(new[]{typeA, singleFolderContainingTypes},
cleanupLibraryMode);
...
```

Modify the following program code to invoke cleanup action for all types in a library:

```
 Copies the following program code to clipboard.
...
ILibrary myLibrary = ...;
CleanupLibraryMode cleanupLibraryMode = ...;
myLibrary.Cleanup(new [] {myLibrary.TypeFolder},
cleanupLibraryMode);
...
```

3.27 Harmonize project from project library

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to harmonize the names and paths of type instances with the types in a specified library. The HarmonizeProject() is available for both project and global libraries. This feature is analogous to the Harmonize feature present in TIA Portal user interface.

You can specifiy which devices in the project to harmonize along with the option to harmonize names, paths, or both. If the HarmonizeProject feature is called on a global library, the project library will be first harmonized and then the project.

The Harmonize is handled according to the chosen HarmonizeProjectOptions:

HarmonizeProjectOptions	Description

HarmonizePaths	Harmonize paths of the specified types or the types under the specified folders. The instances in the project are renamed to the type names of the library
HarmonizeProjectOptions.HarmonizeNames	Harmonize names of the specified types or the types under the specified folders. The folder structure of the library types are applied to the project instances
HarmonizeProjectOptions.HarmonizeNames | HarmonizeProjectOptions.HarmonizePaths	Harmonize names and paths of types selected or the types under a selected folder
HarmonizeProjectOptions.None	This is not to be used and exception is thrown on usage

Introduction

Modify the following program code to perform Harmonize names for a subset of types and folders:

```
 Copies the following program code to clipboard.
...
ILibrary myLibrary = ...;
HarmonizeProjectOptions harmonizeProjectOptions = ...;
ILibraryType typeA = ...; //from myLibrary
ILibraryType typeB = ...; //from myLibrary
LibraryTypeFolder singleFolderContainingTypes = ...; //from myLibrary
IUpdateProjectScope harmonizeProjectScope = ..... //scope of device in project to be harmonized
myLibrary.HarmonizeProject(new[]{typeA, singleFolderContainingTypes}, new[] {harmonizeProjectScope}, HarmonizeProjectOptions.HarmonizeNames);
...
```

Modify the following program code to perform Harmonize paths for a subset of types and folders:

```
Copies the following program code to clipboard.

...
ILibrary myLibrary = ...;
HarmonizeProjectOptions harmonizeProjectOptions = ...;
ILibraryType typeA = ...; //from myLibrary
ILibraryType typeB = ...; //from myLibrary
LibraryTypeFolder singleFolderContainingTypes = ...; //from
myLibrary
IUpdateProjectScope harmonizeProjectScope = ..... //scope
of device in project to be harmonized
myLibrary.HarmonizeProject(new[]{typeA,
singleFolderContainingTypes}, new[] {harmonizeProjectScope},
HarmonizeProjectOptions.HarmonizePaths);
...
```

Modify the following program code to perform Harmonize names and paths for a subset of types and folders:\

```
Copies the following program code to clipboard.

...
ILibrary myLibrary = ...;
HarmonizeProjectOptions harmonizeProjectOptions = ...;
ILibraryType typeA = ...; //from myLibrary
ILibraryType typeB = ...; //from myLibrary
LibraryTypeFolder singleFolderContainingTypes = ...; //from
myLibrary
IUpdateProjectScope harmonizeProjectScope = ..... //scope
of device in project to be harmonized
myLibrary.HarmonizeProject(new[]{typeA,
singleFolderContainingTypes}, new[] {harmonizeProjectScope},
HarmonizeProjectOptions.HarmonizeNames |
HarmonizeProjectOptions.HarmonizePaths);
...
```

Exception Handling

The exception scenarios are stated below:

NULL in sourceTypesAndFolder collection:



```
var projectLibrary = Project.ProjectLibrary;
var type1 = projectLibrary.Types.FindType("Type1"); // if
this statement yields NULL
projectLibrary.HarmonizeProject(new[]{type1}, new[]
{harmonizeProjectScope},
HarmonizeProjectOptions.HarmonizeNames); // throws
EngineeringTargetInvocationException
```

NULL sourceTypesAndFolder collection:



```
projectLibrary.HarmonizeProject(null, new[] {
harmonizeProjectScope },
HarmonizeProjectOptions.HarmonizeNames); // throws
EngineeringTargetInvocationException
```

Empty sourceTypesAndFolder collection



```
projectLibrary.HarmonizeProject(new[]{}, new[]
{harmonizeProjectScope},
HarmonizeProjectOptions.HarmonizeNames); // throws
EngineeringTargetInvocationException
```

NULL in harmonizeProjectScope collection



```
var projectLibrary = Project.ProjectLibrary;
var type1 = projectLibrary.Types.FindType("Type1");
projectLibrary.HarmonizeProject(new[]{type1}, new[] { null },
HarmonizeProjectOptions.HarmonizeNames); // throws
EngineeringTargetInvocationException
```

HarmonizeProjectOptions is None



```
var projectLibrary = Project.ProjectLibrary;
```

```
var type1 = projectLibrary.Types.FindType("Type1");// if this
statement yields NULL
projectLibrary.HarmonizeProject(new[]{type1}, new[] {
harmonizeProjectScope }, HarmonizeProjectOptions.None); //
throws EngineeringTargetInvocationException
```

sourceTypesAndFolder from different library

```
Copies the following program code to clipboard.
var type1 = projectLibrary.Types.FindType("Type1");
var type2 = globalLibrary.Types.FindType("Type2");
projectLibrary.HarmonizeProject(new[]{type1, type2}, new[] {
harmonizeProjectScope }, HarmonizeProjectOptions.None); //
throws EngineeringTargetInvocationException
```

Other cases of EngineeringTargetInvocationException are as below:

- The project is in ReadOnly state
- The Library is in ReadOnly state
- The harmonizeScope are from different Projects
- The harmonizeScope Collection is in ReadOnly state

3.28 Updating Structure during Library Update

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to update or retain the project and global library structure even if there are conflicts. You can also use the TIA Portal Openness to retain or delete unused versions from target library.

The UpdateLibrary() is defined with the following signature on Project Library and Global Library:

public void UpdateLibrary(IEnumerable<ILibraryTypeOrFolderSelection> sourceTypesAndFolders, ILibrary targetLibrary, ForceUpdateMode forceUpdateMode, DeleteUnusedVersionsMode updateMode, StructureConflictResolutionMode structureConflictResolutionMode);

The UpdateLibrary() is overloaded with another definition on Single Types with the following signature :

public void UpdateLibrary(ILibrary targetLibrary, DeleteUnusedVersionsMode updateMode, StructureConflictResolutionMode structureConflictResolutionMode,ForceUpdateMode forceUpdateMode);

The UpdateLibrary() accepts the following parameter definition to perform different operations:

Parameter	Definition
DeleteUnusedVersionsMode	To control whether to delete/retain unused versions from the Target library. The DeleteUnusedVersionMode performs operations based on the ENUM value provided as input: • DeleteUnusedVersionsMode.AutomaticallyDelete : Unused versions will be deleted in the Target Library if there are unused version in target libraries • DeleteUnusedVersionsMode.DoNotDelete : Unused versions will not be deleted in the Target Library if there are unused version in target libraries
StructureConflictResolutionMode	To control whether to update the target library structure, retain the existing structure or to cancel the operation if there is a structural conflict. The StructureConflictResolutionMode performs operations based on the ENUM value provided as input: • StructureConflictResolutionMode.UpdateStructure : Source Library structure will be updated in the Target Library if there are structural differnces between source and target libraries • StructureConflictResolutionMode.RetainStructure : Source Library structure will be same in the Target Library if there are structural differnces between source and target libraries • StructureConflictResolutionMode.CancelIfStructureConflicts : Throws the EngineeringTargetInvocationException and cancel the operation If a structure conflict occurs
ForceUpdateMode	To control whether to set the updated version on the target library as default/latest. The ForceUpdateMode performs operations based on the ENNUM value provided as input:

	• ForceUpdateMode.SetOnlyHigherUpdatedVersionAsDefault : Default version in source library is updated to target and the updated version is marked as default version in target only if source version number is higher than target version number • ForceUpdateMode.ForceSetAnyUpdatedVersionAsDefault : Default version in source library is updated to target as default or latest version, irrespective of version number • ForceUpdateMode.NoDefaultVersionChange : This enum value should be used during update from project library to project where force update option is not applicable

Program code: Single Type Update Library

The UpdateLibrary API is part of Siemens.Engineering.Library.Types.LibraryType interface and will accept a "singleselection" of types.

Modify the program code to perform update libraryUpdateLibrary for a single type from Project Library to Global Library(delete unused version , update existing target library structure and force update target version as default irrespective of version number)

```
Copies the following program code to clipboard.

...
ILibrary myProjectLibrary = Project.ProjectLibrary;
LibraryTypeUserFolder singleFolderContainingTypes =
myProjectLibrary.TypeFolder.Folders.Find("folder1");
ILibraryType typeA =
singleFolderContainingTypes.Types.Find("block1");
// block1 has lower version.
DeleteUnusedVersionsMode deleteUnusedVersionsMode =
DeleteUnusedVersionsMode.AutomaticallyDelete;
StructureConflictResolutionMode
structureConflictResolutionMode =
StructureConflictResolutionMode.UpdateStructure;
ForceUpdateMode forceUpdateMode =
ForceUpdateMode.ForceSetAnyUpdatedVersionAsDefault;
ILibrary globalLibrary = ...;
typeA.UpdateLibrary(globalLibrary, deleteUnusedVersionsMode,
structureConflictResolutionMode, forceUpdateMode);
...
```

Modify the following program code to perform UpdateLibrary for a single type from Project Library to Global Library(do not delete unused version, retain existing target library structure and do not perform force update):

```
Copies the following program code to clipboard.

...
ILibrary globalLibrary = ...;
LibraryTypeUserFolder singleFolderContainingTypes =
globalLibrary.TypeFolder.Folders.Find("folder1");
ILibraryType typeA =
singleFolderContainingTypes.Types.Find("block1");// block1
has lower version.
DeleteUnusedVersionsMode deleteUnusedVersionsMode =
DeleteUnusedVersionsMode.DoNotDelete;
StructureConflictResolutionMode
structureConflictResolutionMode =
StructureConflictResolutionMode.RetainStructure;
ForceUpdateMode forceUpdateMode =
ForceUpdateMode.SetOnlyHigherUpdatedVersionAsDefault;
ILibrary myProjectLibrary = Project.ProjectLibrary;
typeA.UpdateLibrary(myProjectLibrary,
deleteUnusedVersionsMode, structureConflictResolutionMode,
forceUpdateMode);
...
```

Modify the following program code to perform UpdateLibrary for a single type from Global Library to Project Library(do not delete unused version, retain existing target library structure and do not perform force update):

```
Copies the following program code to clipboard.

...
ILibrary globalLibrary = ...;
LibraryTypeUserFolder singleFolderContainingTypes =
globalLibrary.TypeFolder.Folders.Find("folder1");
ILibraryType typeA =
singleFolderContainingTypes.Types.Find("block1"); // block1
has lower version.
```

```
DeleteUnusedVersionsMode deleteUnusedVersionsMode =
DeleteUnusedVersionsMode.DoNotDelete;

StructureConflictResolutionMode
structureConflictResolutionMode =
StructureConflictResolutionMode.RetainStructure;

ForceUpdateMode forceUpdateMode =
ForceUpdateMode.SetOnlyHigherUpdatedVersionAsDefault;

ILibrary myProjectLibrary = Project.ProjectLibrary;

typeA.UpdateLibrary(myProjectLibrary,
deleteUnusedVersionsMode, structureConflictResolutionMode,
forceUpdateMode);

...
```

Modify the following program code to perform UpdateLibrary for a single type from Global Library to Project Library(delete unused version, update existing target library structure and do not perform force update):

```
 Copies the following program code to clipboard.

...

ILibrary globalLibrary = ...;

LibraryTypeUserFolder singleFolderContainingTypes =
globalLibrary.TypeFolder.Folders.Find("folder1");

ILibraryType typeA =
singleFolderContainingTypes.Types.Find("block1"); // block1
has lower version.

DeleteUnusedVersionsMode deleteUnusedVersionsMode =
DeleteUnusedVersionsMode.AutomaticallyDelete;

StructureConflictResolutionMode
structureConflictResolutionMode =
StructureConflictResolutionMode.UpdateStructure;

ForceUpdateMode forceUpdateMode =
ForceUpdateMode.SetOnlyHigherUpdatedVersionAsDefault;

ILibrary myProjectLibrary = Project.ProjectLibrary;

typeA.UpdateLibrary(myProjectLibrary,
deleteUnusedVersionsMode, structureConflictResolutionMode);

...
```

Program code: Multiple Types Update Library

The UpdateLibrary API is part of Siemens.Engineering.Library.ILibrary interface and will accept a "multiselection" of types and type folders.

Modify the following program code to perform UpdateLibrary for a single type from Project Library to Global Library:

```
Copies the following program code to clipboard.

...ILibrary myProjectLibrary = Project.ProjectLibrary;

LibraryTypeUserFolder singleFolderContainingTypes =
myProjectLibrary.TypeFolder.Folders.Find("folder1");

ILibraryType typeA =
singleFolderContainingTypes.Types.Find("block1"); // block1
has lower version.

ForceUpdateMode forceUpdateMode =
ForceUpdateMode.SetOnlyHigherUpdatedVersionAsDefault;

DeleteUnusedVersionsMode deleteUnusedVersionsMode =
DeleteUnusedVersionsMode.DoNotDelete;

StructureConflictResolutionMode
structureConflictResolutionMode =
StructureConflictResolutionMode.UpdateStructure;

ILibrary globalLibrary;

myProjectLibrary.UpdateLibrary(new[] { typeA },
globalLibrary, forceUpdateMode, deleteUnusedVersionsMode,
structureConflictResolutionMode);

...
```

Modify the following program code to perform UpdateLibrary for multiple types from Project Library to Global Library:

```
Copies the following program code to clipboard.

...

ILibrary myProjectLibrary = Project.ProjectLibrary;

LibraryTypeUserFolder singleFolderContainingTypes =
myProjectLibrary.TypeFolder.Folders.Find("folder1");

ILibraryType typeA =
singleFolderContainingTypes.Types.Find("block1"); // block1
has lower version.
```

```
ILibraryType typeB = ...;

ForceUpdateMode forceUpdateMode =
ForceUpdateMode.SetOnlyHigherUpdatedVersionAsDefault;

DeleteUnusedVersionsMode deleteUnusedVersionsMode =
DeleteUnusedVersionsMode.DoNotDelete;

StructureConflictResolutionMode
structureConflictResolutionMode=
StructureConflictResolutionMode.UpdateStructure;

ILibrary globalLibrary;myProjectLibrary.UpdateLibrary(new[] {
typeA, typeB }, globalLibrary, forceUpdateMode,
deleteUnusedVersionsMode, structureConflictResolutionMode);

...
```

Modify the following program code to perform UpdateLibrary for a user folder containing type from Project Library to Global Library:

```
 Copies the following program code to clipboard.

...

ILibrary myProjectLibrary = Project.ProjectLibrary;

LibraryTypeUserFolder singleFolderContainingTypes =
myProjectLibrary.TypeFolder.Folders.Find("folder1");

ForceUpdateMode forceUpdateMode =
ForceUpdateMode.SetOnlyHigherUpdatedVersionAsDefault;

DeleteUnusedVersionsMode deleteUnusedVersionsMode =
DeleteUnusedVersionsMode.DoNotDelete;

StructureConflictResolutionMode
structureConflictResolutionMode =
StructureConflictResolutionMode.UpdateStructure;

ILibrary globalLibrary;

myProjectLibrary.UpdateLibrary(new[] {
(ILibraryTypeOrFolderSelection)singleFolderContainingTypes },
globalLibrary, forceUpdateMode, deleteUnusedVersionsMode,
structureConflictResolutionMode);

...
```

Modify the following program code to perform UpdateLibrary on "Types" system folder from Project Library to Global Library:

```
 Copies the following program code to clipboard.
```

```
...
ILibrary myProjectLibrary = Project.ProjectLibrary;
LibraryTypeSystemFolder typeFolder =
myProjectLibrary.TypeFolder;
ForceUpdateMode forceUpdateMode =
ForceUpdateMode.SetOnlyHigherUpdatedVersionAsDefault;
DeleteUnusedVersionsMode deleteUnusedVersionsMode =
DeleteUnusedVersionsMode.DoNotDelete;
StructureConflictResolutionMode
structureConflictResolutionMode =
StructureConflictResolutionMode.RetainStructure;
ILibrary globalLibrary;
myProjectLibrary.UpdateLibrary(new[] {
(ILibraryTypeOrFolderSelection)typeFolder }, globalLibrary,
forceUpdateMode, deleteUnusedVersionsMode,
structureConflictResolutionMode);
...
```

Modify the following program code to perform UpdateLibrary for a single type from Global Library to Project Library:

```
 Copies the following program code to clipboard.
...
ILibrary globalLibrary = ...;
LibraryTypeUserFolder singleFolderContainingTypes =
globalLibrary.TypeFolder.Folders.Find("folder1");
ILibraryType typeA =
singleFolderContainingTypes.Types.Find("block1"); // block1
has lower version.
ForceUpdateMode forceUpdateMode =
ForceUpdateMode.SetOnlyHigherUpdatedVersionAsDefault;
DeleteUnusedVersionsMode deleteUnusedVersionsMode =
DeleteUnusedVersionsMode.DoNotDelete;
StructureConflictResolutionMode
structureConflictResolutionMode =
StructureConflictResolutionMode.UpdateStructure;
ILibrary myProjectLibrary = Project.ProjectLibrary;
```

```
globalLibrary.UpdateLibrary(new[] { typeA },
myProjectLibrary, forceUpdateMode, deleteUnusedVersionsMode,
structureConflictResolutionMode);

...
```

Modify the following program code to perform UpdateLibrary for a single type from Global Library to Global Library:

```
Copies the following program code to clipboard.

...
ILibrary globalLibrarySource = ...;
LibraryTypeUserFolder singleFolderContainingTypes =
globalLibrarySource.TypeFolder.Folders.Find("folder1");
ILibraryType typeA =
singleFolderContainingTypes.Types.Find("block1"); // block1
has lower version.
ForceUpdateMode forceUpdateMode =
ForceUpdateMode.SetOnlyHigherUpdatedVersionAsDefault;
DeleteUnusedVersionsMode deleteUnusedVersionsMode =
DeleteUnusedVersionsMode.DoNotDelete;
StructureConflictResolutionMode
structureConflictResolutionMode =
StructureConflictResolutionMode.UpdateStructure;
ILibrary globalLibraryTarget = ...;
globalLibrarySource.UpdateLibrary(new[] { typeA },
globalLibraryTarget, forceUpdateMode,
deleteUnusedVersionsMode, structureConflictResolutionMode);
...
```

Exception handling for Single Type Exception Handling

Some of the exception scenarios are stated below:

StructureConflictResolutionMode is CancelIfStructureConflicts

```
Copies the following program code to clipboard.

var myProjectLibrary = Project.ProjectLibrary;
LibraryTypeSystemFolder typeFolder =
myProjectLibrary.TypeFolder.Folders.Find("folder1"); //from
```

```
myProjectLibrary

var type1 = typeFolder.Types.FindType("Type1"); // if this
structure is not available in Global Library

type1.UpdateLibrary(globalLibrary,
DeleteUnusedVersionsMode.DoNotDeleteUnusedVersions,
StructureConflictResolutionMode.CancelIfStructureConflicts,
ForceUpdateMode.SetOnlyHigherUpdatedVersionAsDefault); //
throws EngineeringTargetInvocationException
```

Target Library is NULL.

```
 Copies the following program code to clipboard.

var myProjectLibrary = Project.ProjectLibrary;

var type1 = projectLibrary.Types.FindType("Type1");

ILibrary globalLibrary = Null;

type1.UpdateLibrary(globalLibrary,
DeleteUnusedVersionsMode.DoNotDeleteUnusedVersions,
StructureConflictResolutionMode.RetainStructure,
ForceUpdateMode.SetOnlyHigherUpdatedVersionAsDefault); //
throws EngineeringTargetInvocationException
```

Bad Enum values throws EngineeringTargetInvocationException:

```
 Copies the following program code to clipboard.

var myProjectLibrary = Project.ProjectLibrary;

LibraryTypeSystemFolder typeFolder =
myProjectLibrary.TypeFolder.Folders.Find("folder1");//from
myProjectLibrary

var type1 = typeFolder.Types.FindType("Type1"); // if this
structure is not available in Global Library

type1 .UpdateLibrary(globalLibrary,
(DeleteUnusedVersionsMode)42,
StructureConflictResolutionMode.RetainStructure,
(ForceUpdateMode)22); // throws
EngineeringTargetInvocationException
```

Exception handling for Multiple Type Exception Handling

Some of the exception scenarios are stated below:

NULL in sourceTypesAndFolder collection

```
Copies the following program code to clipboard.

var myProjectLibrary = Project.ProjectLibrary;

var type1 = projectLibrary.Types.FindType("Type1"); // if
this statement yields NULL

myProjectLibrary.UpdateLibrary(new[] { type1 },
globalLibrary,
forceUpdateMode.SetOnlyHigherUpdatedVersionAsDefault,
DeleteUnusedVersionsMode.DoNotDeleteUnusedVersions,
StructureConflictResolutionMode.RetainStructure); // throws
EngineeringTargetInvocationException
```

StructureConflictResolutionMode is CancelIfStructureConflicts

```
Copies the following program code to clipboard.

var myProjectLibrary = Project.ProjectLibrary;

LibraryTypeSystemFolder typeFolder =
myProjectLibrary.TypeFolder.Folders.Find("folder1"); //from
myProjectLibrary

var type1 = typeFolder.Types.FindType("Type1"); // if this
structure is not available in Global Library

myProjectLibrary.UpdateLibrary(new[] { type1 },
globalLibrary,
forceUpdateMode.SetOnlyHigherUpdatedVersionAsDefault,
DeleteUnusedVersionsMode.DoNotDeleteUnusedVersions,
StructureConflictResolutionMode.CancelIfStructureConflicts);
// throws EngineeringTargetInvocationException
```

Bad Enum values throws EngineeringTargetInvocationException

```
Copies the following program code to clipboard.

var myProjectLibrary = Project.ProjectLibrary;

LibraryTypeSystemFolder typeFolder =
myProjectLibrary.TypeFolder.Folders.Find("folder1"); //from
myProjectLibrary

var type1 = typeFolder.Types.FindType("Type1"); // if this
structure is not available in Global Library

myProjectLibrary.UpdateLibrary(new[] { type1 },
globalLibrary, (ForceUpdateMode)22,
(DeleteUnusedVersionsMode)42,
```

```
StructureConflictResolutionMode.RetainStructure); // throws
EngineeringTargetInvocationException
```

Note

Other cases in which EngineeringTargetInvocationException is thrown are as below:

- Any Update Library operations targeting a read-only global library will result in an exception being thrown.
- An exception will be thrown if any of the selected types or folders are null or not part of the source library.

See also

Opening a project

3.29 Updating libraries type

Requirement

- The application is connected to the TIA Portal using TIA Portal Openness
  See Connecting to the TIA Portal
- The project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to opt out a type from updating to project library or project from global library. The feature helps you to update project or project library with types that need an update and ignore types that are not required.

The feature also helps you to distribute linked libraries from different vendors and update project or project library when the vendor provides additional functionality and is not waiting for entire library to be delivered by all the vendors.

You can use the SetForUpdate attribute only for non write protected global library types where as Get operation can be performed on all the library types. The SetForUpdate always returns True for types in project library.

The following API is used to mark the type which are required to be updated to project library or project.

Attribute name	Service Provider	Description
SetForUpdate	LibraryType	To set or get the distributable property of a

		Library Type

Note

The Siemens.Engineering.EngineeringNotSupportedException will be thrown to you if you try

- To perform a set operation on SetForUpdate attribute for types in project library
- To perform a set operation on SetForUpdate attribute for types which are write protected

Program code

Modify the following program code to mark the library type for update using SetForUpdate attribute:

```
Copies the following program code to clipboard.
using Siemens.Engineering;
...
{
     TiaPortal portal = new TiaPortal(TiaPortalMode.WithUserInterface);
     string globalLibrrayPath =
@"C:\Proj\GlobalLibrary_SetForUpdate\GlobalLibrary_SetForUpdate.al18";
     FileInfo globalLibraryFileInfo = new FileInfo(globalLibrrayPath);
     var gl = portal.GlobalLibraries.Open(globalLibraryFileInfo,
OpenMode.ReadWrite);
     var globalLibTypes = gl.TypeFolder.Types;
     //Get 'SetForUpdate' value for a specific library type
     Console.WriteLine(string.Format(":::GloblaLibrary:::SetforUpdateVal
{0}", globalLibTypes[0].SetForUpdate.ToString()));
     //Set 'SetForUpdate' value for a specific library type
      globalLibTypes[0].SetForUpdate = true;
}
```

3.30 Deleting library content

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project
- You have access to the required library.
  See Accessing global libraries.
- You have access to a folder for types.
  See Accessing folders in a library.

Introduction

You can delete the following project library content using the TIA Portal Openness API interface:

- Types
- Type version
- User-defined folders for types
- Master copies
- User-defined folders for master copies

Note

Deleting of types and user-defined type folders

If you want to delete a type or user-defined folder type, the "Rules for deleting versions" must be met. You can always delete an empty type folder.

Note

Rules for deleting versions

You can only delete versions with Committed status. The following rules also apply when deleting versions:

- If a new version with the InWork status has just been created from a version with Committed status , you can only delete the version with Committed status when the new version is discarded or it obtains the "Committed" status.
- If a type only has one version, the type is deleted as well.
- If Version A is dependent on Version B of another type, first delete Version A and then Version B.
- If there are instances of Version A, you can only delete Version A if the instances are deleted as well. If an instance is also contained in a master copy, the master copy is deleted as well.

Program code

Modify the following program code to delete types or user-defined type folders:

```
Copies the following program code to clipboard.
public static void
DeleteMultipleTypesOrTypeUserFolders(ILibrary library)
{
    LibraryType t1 = library.TypeFolder.Types.Find("type1");
    LibraryType t2 = library.TypeFolder.Types.Find("type2");
    LibraryTypeUserFolder f1 =
library.TypeFolder.Folders.Find("folder1");
    t1.Delete();
    t2.Delete();
    f1.Delete();
}
```

Modify the following program code to delete an individual type or user-defined type folder:

```
Copies the following program code to clipboard.
public static void DeleteSingleTypeOrTypeUserFolder(ILibrary
library)
{
    //Delete a single type
    LibraryType t1 = library.TypeFolder.Types.Find("type1");
    t1.Delete();

    //Delete a single folder
    LibraryTypeFolder parentFolder = library.TypeFolder;
    LibraryTypeUserFolder f1 =
parentFolder.Folders.Find("folder1");
    f1.Delete();
}
```

Modify the following program code to delete a version:

```
Copies the following program code to clipboard.
public static void DeleteVersion(ILibrary library)
{
    LibraryType singleType =
library.TypeFolder.Types.Find("type1");
    LibraryTypeVersion version1 =
singleType.Versions.Find(new System.Version(1, 0, 0));
    version1.Delete();
}
```

Modify the following program code to delete a master copy or a user-defined master copy folder:

```
Copies the following program code to clipboard.
public static void DeleteMasterCopies(ILibrary library)
{
    // Delete master copy
    MasterCopy masterCopy =
library.MasterCopyFolder.MasterCopies.Find("myMasterCopy");
    masterCopy.Delete();

    // Delete master copy user folder
    MasterCopyUserFolder masterUserFolder =
library.MasterCopyFolder.Folders.Find("myFolder");
    masterUserFolder.Delete();
}
```

See also

Accessing master copies

3.31 Setting default version of a type

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal

- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Introduction

You can use the TIA Portal Openness to set any released version as default version of that type for project library and user global library.

Default version is considered as preferred version of a type.

Program code



```
Copies the following program code to clipboard.

ILibrary myLibrary = ....;

LibraryType typeA = ...;// from myLibrary

LibraryTypeVersion versionA = ...;//from typeA

versionA.setAsDefault(); // set the version as a default
version of the type.
```

Exception Handling

An exception "EngineeringTargetInvocationException" will be thrown to you if a failure happened during the invocation of SetAsDefault API.

You can encounter exception in the following scenarios:

- In project library, when SetAsDefault() is called on in-test/in-work version
- When Global Library is read-only

See also

Opening a project

3.32 Getting default version of a type

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Introduction

You can use the TIA Portal Openness to get default version of a type for project library and global library. You can also use the TIA Portal Openness application to check if a version is default or not. Default version is considered as preferred version of a type.

Program code

```
Copies the following program code to clipboard.
//...
ILibrary myLibrary = ...;
LibraryType typeA = ...// from myLibrary
var typeADefaultVersion = typeA.Versions.First((version) =>
version.IsDefault);
//from default version , user can get details related to
version like Author, Guid, etc.
var nameofType = typeADefaultVersion.LibraryTypeName;
var guid = typeADefaultVersion.Guid;
var versionNumber = typeADefaultVersion.VersionNumber;
var authoroftheVersion = typeADefaultVersion.Author;
var status = typeADefaultVersion.State;
//...
```

See also

Opening a project

3.33 Improved types consistency state

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Introduction

You can use the TIA Portal Openness to check types for a inconsistencies. The Types can have different consistency status. The Type is Consistent, if Dependent Type is using default version of dependency. The Type is Inconsistent if Dependent Type is not using the default version of dependency type.

The folder level consistency status shows if any type under the structure is consistent or inconsistent.

The Status API is part of Siemens.Engineering.Library.Types.LibraryTypeand Siemens.Engineering.Library.Types.LibraryTypeFolderclass and will be of ConsistencyStatustype.

To allow TIA Portal Openness user to check the consistency status of type/folders a "Status" attribute will be introduced for the following:

- SystemFolder
- UserFolder
- Type

Using this, you can prepare the list of inconsistent types to perform special operations on them:



```
var listOfInconsistentTypes = Folder.Types.Where(x =>
x.Status == ConsistencyStatus.DefaultVersionInconsistent);
```

ConsistencyStatus enum supports below mentioned values:

Value	Description
DefaultVersionInConsistent	Dependent Types are not using the default version of Dependency Types
Consistent	Dependent Types are using the defautt version of Dependency Types

Program code

Modify the following program code to read status attribute support on "Types" system folder from Project Library:



```
...
ILibrary myProjectLibrary = Project.ProjectLibrary;
LibraryTypeSystemFolder typeFolder =
myProjectLibrary.TypeFolder;
var systemTypeFolderConsistencyStatus = typeFolder.Status;
if(systemTypeFolderConsistencyStatus ==
ConsistencyStatus.DefaultVersionInConsistent)
{
```

```
//Do something.
}
// List of InConsistentType
var listOfInCosistentTypes = typeFolder.Types.Where(x =>
x.Status == ConsistencyStatus.InConsistent);
foreach(var type in listOfInCosistentTypes)
{// From type, user can get details like Author, Guid etc
var nameOfType = type.LibraryTypeName;
var guid = type.Guid;
var authorOfTheVersion = type.Author;
var status = type.State;
}
...
```

Modify the following program code to read Status Attribute on "User Folder" from Project Library:

```
 Copies the following program code to clipboard.
...
ILibrary myProjectLibrary = Project.ProjectLibrary;
LibraryTypeUserFolder userFolder =
myProjectLibrary.TypeFolder.Folders.Find("folder1");
var userTypeFolderConsistencyStatus = userFolder.Status;
if(userTypeFolderConsistencyStatus ==
ConsistencyStatus.DefaultVersionInConsistent)
{
//Do something.
}
...
```

Modify the following program code to read Status Attribute on "Individual Type" from Project Library:

```
 Copies the following program code to clipboard.
...
ILibrary myProjectLibrary = Project.ProjectLibrary;
```

```
LibraryTypeUserFolder userFolder =
myProjectLibrary.TypeFolder.Folders.Find("folder1");
ILibraryType blockType = userFolder.Types.Find("block1");
var blockTypeConsistencyStatus = blockType.Status;
if(blockTypeConsistencyStatus ==
ConsistencyStatus.DefaultVersionInConsistent)
{
//Do something.
}
...
```

Note

This attribute can also be used in similar way in Global Library.

Exception Handling

An ArgumentNullException exception will be thrown to you if any of the selected types or folders are null or not part of the source library.



```
Copies the following program code to clipboard.
var projectLibrary = Project.ProjectLibrary;
var type1 = projectLibrary.Types.FindType("Type1");
var status = typeFolder.Status; // throws
ArgumentNullException
```

See also

Opening a project

3.34 Editing library type version

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness application to edit a library type version within project. To edit a library type version, you can use the Edit() method on LibraryTypeVersion.

The Edit() method returns LibraryTypeVersion, which is of type Siemens.Engineering.Library.Types.LibraryTypeVersion.

In Step 7, the returned version is always an in-work or in-test type created from a selected existing version and is used for types that require an instance to edit the type, while in HMI, the returned version is always a new in-work or in-test type created from a selected version and is used for types that do not require an instance to edit the type.

Note

The Edit() method is available in Project Library and not supported via Global Library

Parameter

The Edit() method in Step7 accepts the following parameter:

Parameter name	Data types	Mandatory	De
libraryTypeInstanceInfo	Siemens.Engineering.Library.Types.LibraryTypeInstanceInfo	Yes	Ins inf for ver be

Note

The Edit() method is a method overload that accepts no parameter for HMI.

Exception

The Edit() method will throw EngineeringTargetInvocation exception in the following conditions:

- Version already in-work
- Any version of same type is in-work
- Edit version is not allowed due to write protected or read only persistence
- Version is in global library
- Version related required packages are missing

Program code

To edit a library type version in Step7 modify the following program code:



```
Copies the following program code to clipboard.

private static void EditLibraryTypeStep7(TiaPortal portal)
{
      var project = portal.Projects.Open(new
System.IO.FileInfo
      (@"D:\Temp\Test\Project1_test2\Project1_test2.ap20"));
      LibraryTypeComposition libTypes =
project.ProjectLibrary.TypeFolder.Types;
      try
      {
      var libraryVersion = libTypes[0].Versions[0];
      var device = project.Devices[0];
      var rack = device.Items[0];
      var plc = rack.Items[0];
      var softwareContainer =
plc.GetService<SoftwareContainer>();
      var plcSoftware = softwareContainer?.Software as
PlcSoftware;
      IInstanceSearchScope searchScope = plcSoftware as
IInstanceSearchScope;
      IList<LibraryTypeInstanceInfo> libraryTypeInstanceInfos
= libraryVersion.FindInstances(searchScope);
      if(libraryTypeInstanceInfos.Any())
      {
      var libraryInWorkVersion =
libraryVersion.Edit(libraryTypeInstanceInfos.First());
      }
      }
      catch (EngineeringException e)
      {
```

```
      Console.WriteLine(e);
      }
}
```

To edit a library type version in HMI modify the following program code:

```
Copies the following program code to clipboard.
private static void EditLibraryTypeHMI(TiaPortal portal)
{
      var project = portal.Projects.Open(new
System.IO.FileInfo
      (@"D:\Temp\Test\Project1_test2\Project1_test2.ap20"));
      LibraryTypeComposition libTypes =
project.ProjectLibrary.TypeFolder.Types;
      try
      {
      var libraryVersion = libTypes[0].Versions[0];
      var libraryInWorkVersion = libraryVersion.Edit();
      }
      catch (EngineeringException e)
      {
      Console.WriteLine(e);
      }
}
```

3.35 Discarding library type version

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness application to discard library type version within TIA Portal project.

To discard the library type version, you can use the Discard() method on libraryVersionInWork.

Method

Method name	Description
Discard()	Delete the edited version. Reset all uses of the deleted version to last released version.

Exception

The Discard() will throw EngineeringTargetInvocation exception in the following conditions:

- Discard of version not in-work
- Discard of version with missing packages

Program code

```
Copies the following program code to clipboard.
private static void DiscardingLibraryTypeVersion(TiaPortl portal)
{
      Project project = portal.Projects.Open(new
      System.IO.FileInfo(@"D:\Temp\Test\Project1_test2\Project1_test2.ap
      LibraryTypeComposition libTypes =
project.ProjectLibrary.TypeFolder.Types;
      try
      {
      //Client gets the desired type
      in-work version that needs to be discarded.
      LibraryTypeVersion libraryVersionInWork =
GetInWorkVersionToDiscard(libTypes);
      libraryVersionInWork.Discard();
      }
      catch (EngineeringException e)
```

```
        {
        Console.WriteLine(e);
        }
}
```

3.36 Managing alarm text list in PLC and Mastercopy library

Requirement

- The TIA Portal Openness is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to copy PLC text lists to a library as Master copy or delete PLC text lists from library Master copy folders. Both project and global libraries can be used.

You can also use the TIA Portal Openness to support the following functions :

- Accessing user text lists and systemtext lists
- Updating user text lists
- Deleting user text lists
- Setting text lists name and ListRange
- Accessing information about a textlist including ID, name, and type for user or system

Property

Reference of PlcAlarmTextlistGroup (Navigators)

Custom type	Navigator	Data type	Description
PlcUnitBase	PlcAlarmTextlistGroup	PlcAlarmTextlistGroup	It is the first level of text lists and creates the EOM classes of System and User text lists.
PlcSoftware			

Navigators of PlcAlarmTextlistGroup

Navigators	Datatype

PlcAlarmSystemTextlists	PlcAlarmSystemTextlistComposition
PlcAlarmUserTextlists	PlcAlarmUserTextlistComposition

PlcAlarmTextlist

The following properties are supported in PLC alarm text lists:

Property name	Data type	Description	Access
ID	Int32	Get id of the PLC alarm text lists	Read
ListRange	PlcAlarmTextlistListRange	Get types of range for PLC alarm text lists. The PlcAlarmTextlistListRange can accept the following enum values: • Decimal: Decimal value of a range type • Binary: Binary value of a range type • Bit: Bit value of a range type • None: None value of a range type	Read
Name	String	Get name of the PLC alarm text lists	Read

Navigator of PlcAlarmTextlist

Navigator	Data type
Comment	MultilingualText

PlcAlarmUserTextlist

Property name	Data type	Description	Access
ListRange	PlcAlarmTextlistListRange	Get the types of range for PLC alarm text lists	Read/Write

Name	String	Get the name of the PLC alarm text lists	Read/Write

Program code



```
Copies the following program code to clipboard.

TiaPortal tia = TiaPortal.GetProcesses().Select(x => x.Attach()).First()
Project project = tia.Projects.First();
Device device = project.Devices.First();
DeviceItem deviceItem = (DeviceItem)(from x in device.DeviceItems where
x.Name.Equals("PLC_1") select x).First();
SoftwareContainer softwareContainer =
((IEngineeringServiceProvider)deviceItem).GetService<SoftwareContainer>
PlcSoftware plc = softwareContainer.Software as PlcSoftware;
var group = plc.PlcAlarmTextlistGroup;
var groupParent = group.Parent;
var systemTextlist = group.PlcAlarmSystemTextlists.First();
var listRangeType = systemTextlist.ListRange;
var systemTextlistParent = systemTextlist.Parent;
var userTextlist = group.PlcAlarmUserTextlists.First();
var userTextlistParent = userTextlist.Parent;
var listRange = systemTextlist.ListRange;
var comment = systemTextlist.Comment;
userTextlist.Name = "set text";
userTextlist.ListRange =
Siemens.Engineering.SW.Alarm.TextLists.PlcAlarmTextlistListRange.Bit;
userTextlist.Comment.Items.Find(project.LanguageSettings.ActiveLanguages
CultureInfo("en-US", false))).Text = "EN comment";
userTextlist.Comment.Items.Find(project.LanguageSettings.ActiveLanguages
CultureInfo("de-DE", false))).Text = "DE comment";
userTextlist.Delete();
MasterCopy masterCopy =
project.ProjectLibrary.MasterCopyFolder.MasterCopies.First<MasterCopy>(x
x.Name == "textlist_in_library");
```

```
PlcAlarmUserTextlists.CreateFrom(masterCopy);
masterCopy.Delete();
project.ProjectLibrary.MasterCopyFolder.MasterCopies.Create(userTextlist
```

3.37 Managing DoNotUse in Library

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness application to mark or unmark the DoNotUse attribute for any library type within project library or global library.

Note

You need to iterate through multiple library types or folders to mark or unmark the DoNotUse attribute.

Program code

To mark or unmark the 'DoNotUse' attribute for a library type modify the following program code:

```
Copies the following program code to clipboard.
private static void DoNotUseFeature(TiaPortal tiaPortal)
{
    var project = tiaPortal.Projects.Open
   (new
System.IO.FileInfo(@"D:\Temp\Test\Project1_test2\Project1_test2.ap19"))
    LibraryTypeComposition libTypes =
project.ProjectLibrary.TypeFolder.Types;
    try
    {
    //Mark DoNotUse
```

```
    libTypes[0].DoNotUse = true;

    //Unmark DoNotUse
    libTypes[0].DoNotUse = false;
    }
    catch (EngineeringException e)
       {
            Console.WriteLine(e);
       }
}
```

3.38 Exporting library type version as document

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to create a document from library type to distribute standard software and manage source control. You can use the ExportAsDocuments() method in the TIA Portal Openness application to create a document from a specific library type.

Note

The TIA Portal Openness application supports this feature in the project library, but not in the global library.

Parameter and Enum

To create a document from the library type, the ExportAsDocuments() method uses the following parameter options:

Parameter name	Data type	Mandatory	Description
directoryInfo	System.IO.DirectoryInfo	Yes	Specifies the export file

			directory information
fileNameWithoutExtension	System.String	Yes	Specifies the export file name without extension
exportFormat	System.String	Yes	Specifies the export format
libraryExportOptions	Enum	Yes	Specifies the export options to be used to control LibraryVersionInfo file and domain file generation

The libraryExportOptions parameter uses the following enum options:

Enum name	Description	Example
LibraryExportOptions.None	The library version info file is not exported , only the domain specific language files are exported.	myBlockType.Versions[0].ExportAsDocu DirectoryInfo("C:\\Temp"), "Block_1", "SimaticMLWithExportOptionsWithDefau LibraryExportOptions.None);
LibraryExportOptions.WithLibraryVersionInfoFile	Both library version info file and Domain specific langauge files are exported.	myBlockType.Versions[0].ExportAsDocu DirectoryInfo("C:\\Temp"), "Block_1", "SCI LibraryExportOptions.WithLibraryVersion
LibraryExportOptions.OnlyLibraryVersionInfoFile	Only the library version info file is exported , no domain specific	

	language files are exported.	myBlockType.Versions[0].ExportAsDocuments(new DirectoryInfo("C:\\Temp"), "Block_1", "S7DCL", LibraryExportOptions.OnlyLibraryVersionInfoFile);

The ExportAsDocuments() returns ExportTransferResult that contains list of following details:

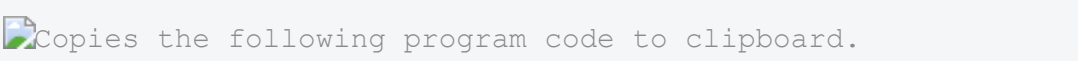
Type	Description
ExportTransferResult.Messages	Browse through collection of log messages generated during version import. Messages is TransferResultMessageComposition that is composed of list of TransferResultMessage. TransferResultMessage has log message details.
ExportTransferResult.ExportedDocuments	Returns the list of Exported Documents. ExportedDocuments is list of FileInfo.
ExportTransferResult.TransferResultState	Detailed information on state of the create version from document. TransferResultState is Enum has Success or Warning. An error condition will be always returned as an Engineering Exception, which is not represented by this enum value.
ExportTransferResult.Parent	EOM parent of this object. Parent is IEngineeringObject.

Exception

The EngineeringTargetInvocation exception will be thrown to you in the following cases:

- If you have not passed any of the mandatory parameters in the ExportAsDocument()
- If you are trying to export write-protected type
- If you are trying to invalid export format
- If you already have a directory containing an exported file or a file with a required extension that matches the given filename

Program code



```
private static void
ExportLibraryTypeVersionAsDocument(TiaPortal portal)
{
    var project = portal.Projects.Open(new System.IO.FileInfo
    (@"D:\Temp\Test\Project1_test2\Project1_test2.ap20"));
    LibraryTypeComposition libTypes =
project.ProjectLibrary.TypeFolder.Types;
    try
    {
    var supportedFormats =
libTypes[0].GetSupportedExportFormats();
    var libraryVersion = libTypes[0].Versions[0];
    DirectoryInfo directoryInfo = new
DirectoryInfo(@"D:\OpenessExportTest");
    ExportTransferResult exportTransferResult =
libraryVersion.
    ExportAsDocuments(directoryInfo, libTypes[0].Name,
supportedFormats[0], LibraryExportOptions.None);
    if(exportTransferResult.TransferResultState ==
TransferResultState.Success)
    {
    IEnumerable<FileInfo> exportedFiles =
exportTransferResult.ExportedDocuments;
    Console.WriteLine("Version exported successfully");
    }
    else
    {
    Console.WriteLine("Version export failed.");
    foreach (var transferResultMessage in
exportTransferResult.Messages)
    {
     Console.WriteLine(transferResultMessage.Message);
    }
```

```
    }
    }
    catch (EngineeringException)
    {
    Console.WriteLine(e);
    }
    }
}
```

3.39 Retrieving supported export formats for library

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to get all supported document formats for a specific library types. You can use these document formats to export library type version.

The following document formats are supported while using the TIA Portal Openness application:

- Simatic ML
- WinCCML
- S7DCL
- SCL
- STL
- UDT

Note

The TIA Portal Openness application supports this feature in the project library, but not in the global library.

Method

You can use the GetSupportedExportFormats() method in TIA Portal Openness to get all the supported document for export for a specific library type.

Return type	Description
IEnumerable<string> GetSupportedExportFormats()	Returns a list of string of supported document formats for successful execution

Exception

No specific exception.

Program code

To get all supported document format to export modify the following program code:

```
Copies the following program code to clipboard.
private static void RetrieveSupportedDocumentFormat(TiaPortal portal)
{
    var project =
portal.Projects.Open((@"D:\Temp\Test\Project1_test2\Project1_test2.ap20"
    LibraryTypeComposition libTypes =
project.ProjectLibrary.TypeFolder.Types;
    try
    {
    var supportedFormats = libTypes[0].GetSupportedExportFormats();
    }
    catch (EngineeringException e)
    {
     Console.WriteLine(e);
    }
}
```

3.40 Releasing library type version

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal

- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to release a new library type version in a project. You can use the Release() method to create a compile clean new version for a library type to be used in a project.

Parameter and Enum

To release a new version of a library type, the Release() method uses the following parameter options:

Parameter name	Data type	Mandatory	Description
createOrReleaseDependenciesMode	Enum	Yes	Options used to control whether or not the operation will proceed if it will affect dependencies
versionNumber	Version	No	Specifies the version number
author	string	No	Specifies the author name about the version
comment	string	No	Specifies the comment information about the version

The createOrReleaseDependenciesMode parameter uses the following enum options:

Enum name	Description
AutomaticallyCreateOrReleaseDependenciesIfRequired	Automatically create or release any dependencies if required

DoNotAutomaticallyCreateOrReleaseDependencies	If dependencies must be created or released, abort the operation

Exception

The EngineeringTargetInvocation exception will be thrown to you in the following cases:

- If your type version is failed to get released

Program code

To release a library type version modify the following program code:



```
Copies the following program code to clipboard.
private static void ReleasingLibraryTypeVersion(TiaPortal portal)
{
    var project = portal.Projects.Open
   (new
System.IO.FileInfo(@"D:\Temp\Test\Project1_test2\Project1_test2.ap20"));
    var directoryInfo = new DirectoryInfo(@"D:\Temp\Test");
    const string fileName = "TestFile";
    var device = project.Devices[2];
    var rack = device.Items[0];
    var plc = rack.Items[0];
    var softwareContainer = plc.GetService<SoftwareContainer>();
    var plcSoftware = softwareContainer?.Software as PlcSoftware;
    PlcBlockSystemGroup blockGroup = plcSoftware.BlockGroup;
    try
    {
    var type = project.ProjectLibrary.TypeFolder.Types[1];
    var versionComposition = type.Versions;
    VersionCreateTransferResults versionCreateTransferResult =
    versionComposition.CreateFromDocuments(directoryInfo, fileName,
blockGroup, CreateOptions.Override);
    // in-work version object created
```

```
    if(versionCreateTransferResult.TransferResultState ==
TransferResultState.Success)
    {
    LibraryTypeVersion versionInWork =
versionCreateTransferResult.CreatedVersion;
    Version versionNumber = new Version(0,0,1);
    string author = "z00test";
    string comment = "test";
    versionInWork.Release(CreateOrReleaseDependenciesMode.
    AutomaticallyCreateOrReleaseDependenciesIfRequired,
    versionNumber, author, comment);
    Console.WriteLine("Version created and released
successfully");
    }
    else
    {
    Console.WriteLine("Version creation failed.");
    foreach (var transferResultMessage in
versionCreateTransferResult.Messages)
    {
    Console.WriteLine(transferResultMessage.Message);
    }
    }
    project.Save();
    project.Close();
    }
    catch (EngineeringException e)
    {
    Console.WriteLine(e);
    }
}
```